June 2026

OpenWorld

VERIFIED CODE WORLD MODELS

build · optimize · deploy

World-Time Compute with Verified Code World Models

James Schwoebel[1,*] • Ingrida Semenec, PhD[1] • Jenia Rousseva, PhD[1] • Marcos Ortiz, PhD[1] • Collin Overbay[1] • Manish Bhatt[4,6] • Rome Thorstenson[3] • Martin G. Frasch, MD, PhD[2,5]

[1] Quome, Inc. [2] University of Washington [3] Rafter Labs, Inc. [4] OWASP [5] Health Stream Analytics, LLC
[6] IEEE Computer Society [*] Corresponding author

ABSTRACT

Large language models generalize across a domain only after seeing many real, labeled examples of it, and in most domains that experience is scarce. We study a way to manufacture it cheaply. When a domain's dynamics can be written as code, one template instantiates into many *world models*—simulators whose dynamics are executable, verifiable programs over symbolic state—each an inexhaustible source of *exactly*-labeled trajectories. Fine-tuning an LLM on trajectories traversed through many such world models of a domain—which we call **world-time compute**, a training-time analogue of test-time compute—lifts its generalization to *held-out* worlds it never trained on (demonstrated on synthesized world families), with the largest gains where model capability is scarcest (+29 points at 0.5B; the largest model's lift is within noise, consistent with task saturation). The labels can be trusted because the worlds are *verified code*, not learned approximations: synthesized-then-checked dynamics are exact over 20-step rollouts and answer 10× out-of-distribution probes exactly (100%), whereas per-step LLM prediction and a trained MLP compound error and collapse. What separates this from domain randomization is that each world is independently authored and individually verified—not parameter variation of one simulator—and a corrupted-label control shows it is label *exactness*, not task variety, that drives the held-out gains. On real, human-authored benchmarks (ARC-AGI grids, List Functions, CLRS algorithms) the same lever holds as *per-world* test-time training, scaling with compute and collapsing under label corruption. On a coherent real family (List Functions) the harder *cross-world* form holds too: one adapter trained on 128 *disjoint* worlds generalizes to held-out worlds it never saw at 40% versus 6% for a corrupted-label control (a +34-point exactness-gated lift, CI [29, 39]). The gain follows a *descriptive* regularity (a saturating curve, not a law): it is largest for *few-step* reasoning and small/weak models, and fades for long reasoning chains, for tasks induced from rich perception, and as a task saturates; cross-task transfer is weak where worlds share no skill. The worlds are authored and served by **OpenWorld**, a zero-dependency Python framework that makes assembling the *many* worlds a domain needs cheap; the framework itself—declaration, the plan–generate–verify relay, composition, perception, and serving—is the subject of a companion paper. **The scope is the symbolic-state regime**: where dynamics are writable, verified code makes a *correct* world model a zero-training prototype with no compounding error; pixel-native domains remain the territory of learned models. All

code, the 100 recipes, experiments, and this manuscript regenerate from one repository.

Keywords: world models; code world models; neuro-symbolic AI; program synthesis; value alignment; agents-as-a-judge; local LLMs.

Code & data: `github.com/quome-cloud/openworld`
Correspondence: jim@quome.com

1 Introduction

Manufacturing experience. A language model generalizes across a domain only after seeing many real, labeled examples of it—and in most domains that experience is scarce and costly. We study a way to manufacture it cheaply. When a domain's dynamics can be written as code, a single template instantiates into many *world models*—simulators whose dynamics are executable, verifiable programs over symbolic state—each an inexhaustible generator of *exactly*-labeled trajectories. Fine-tuning a model on trajectories traversed through many such worlds of a domain—*world-time compute* (§4.3), a training-time analogue of test-time compute—lifts its generalization to *held-out* worlds it never trained on. Two things make this work, and the paper is organized around them. First, the worlds must not lie: a traversed world teaches the right rules only if its dynamics are correct, so we build them as *verified code*, not learned approximations—exact over long rollouts with no compounding error (Figure 1A). Second, world-time compute needs *many* worlds, which is practical only if each is cheap to build; the enabling artifact is OpenWorld, a zero-dependency framework that turns authoring a verified world model (perceive → world → emit → act, served and shareable) into a minutes-scale, push-button activity (the companion framework paper [35]). PyTorch did not give deep learning new models; it made them fast to *prototype*, and an explosion followed—world models have lacked that tool, and we use it here to manufacture training experience.

A world model is an agent's internal simulator: given a state and an action, it predicts the next state (and/or observation), letting the agent plan "in imagination" rather than by trial in the world [15]. That *genus* is shared across the field; research programs diverge on *how the model represents state and how it is obtained*, and the term today names two largely disjoint *species*. The dominant, most visible one is *perceptual and generative*: a neural network learned from large amounts of data whose state is a continuous latent—or pixels, video, or a 3D scene—and whose output is a *generated observation*. It runs from the V–M–C triad of Ha and Schmidhuber [15] through the RSSM-based Dreamer family [16], value-equivalent planning in MuZero [34], autoregressive token models such as IRIS [24] and Δ-IRIS [25], and diffusion dynamics in DIAMOND [3], joint-embedding predictive video models that forecast in representation space such as V-JEPA [22]; and, most prominently in recent generative AI, to interactive generative environments and "spatial intelligence"—internet-scale video and 3D world generators such as Genie [5], video-generation-as-simulator (Sora) [31], and Fei-Fei Li's World Labs [12]. Powerful as these are, they share three structural costs: they are *data-hungry* (millions of environment steps, or internet-scale video), *compounding* (every learned transition adds error that accumulates over a rollout), and *opaque* (dynamics and values alike live in continuous weights that cannot be inspected, edited, or audited).

What "world model" means in this paper. Because the term has come to connote those generative, pixel- and 3D-native systems, we state our usage plainly. We use *world model* in its original, narrower sense—a forward predictor of the next *state* for planning—and we restrict the state to a *symbolic* object: a typed record of named variables (counts, prices, positions, flags), not pixels, latents, or 3D geometry. Our world models are therefore the second species, *Code World Models* (CWMs): the transition is an explicit, human-readable *program* over that symbolic state, synthesized by an LLM from a declared rule set and verified before use, and a rollout returns the next symbolic state *exactly* rather than a generated image, with *zero* training data. This is the same genus as the perceptual models above (a simulator the agent rolls forward) but a different species—and an intentionally complementary one. OpenWorld targets domains that can be *specified* symbolically (inventories, markets, schedules, negotiations, physics with known equations); it is *not* a pixel-native, perceptual, or open-world-3D model and does not generate images, video, or scenes. Where the spatial-intelligence systems *learn what cannot be told*, OpenWorld *writes down what can*

be told, and verifies it; the two are better read as adjacent tools than as competitors.

World, action, agent, simulation—and why a *verified* model is special. The world model is only one of four distinct objects, and keeping them separate dissolves much of the remaining confusion. A *world* is the environment-as-simulator: a symbolic state, a discrete set of *actions*, and the verified transition that maps a (state, action) pair to the next state (plus optional perception/emit boundaries). An *agent* is a *separate* policy—an LLM planner or a fixed rule—that chooses actions toward a goal; it is a *user* of a world, not a part of it, so one world serves any agent and a planner can be swapped without re-synthesizing dynamics. A *simulation* closes the loop: it places agents in a world and repeatedly observes, chooses an action, and steps the world, producing a trajectory scored by objectives. The textbook definition puts the world model *inside* the agent (the simulator it plans with) and distinguishes it from the true environment; the gap between the two—model bias, sim-to-real error—is exactly what bounds learned world models. A verified code world model does not so much eliminate that gap as *relocate* it: once the dynamics are declared symbolically and the accepted program reproduces them exactly, the agent's internal model and the (symbolic) environment are the same artifact, so the dynamics contribute *no* rollout error—what error remains lives entirely in the *specification* and in *perception* (the symbol-grounding boundary; see the companion framework paper [35]), not in the transition. This is why the same `World` serves, without modification, both as the ground-truth environment and as the model an agent plans through in our planning experiments; it is also why this paper is scoped to *specifiable* domains, where that relocation is a genuine win.

This Code World Model bargain—programs instead of weights—buys verifiability, compute-cheap rollouts, and out-of-distribution generalization by construction [40, 11, 32, 18]. Yet CWM research has remained a set of bespoke systems aimed at particular benchmarks. There has been no general, lightweight framework that lets a practitioner *declare* a world, have a local model write and verify its dynamics, steer behavior through declared value weights, and optimize the whole configuration against a goal—the workflow that made supervised learning ubiquitous.

OpenWorld is that framework, and this paper benchmarks the paradigm it operationalizes against the learned-dynamics alternative. The framework contributes four mechanisms: (i) a *plan–generate–verify relay* in which a generator LLM writes transition code that must survive syntactic checks, sandboxed smoke-runs, declared invariants, and optionally a second-model critic before acceptance; (ii) *open value specification*—objectives are scoring functions weighted by tunable dials, so moral configuration is an editable artifact rather than a frozen weight, directly addressing the specification trap [38]; (iii) an *automated tuner* that searches world design, policy knobs, and moral dials jointly (random, local-refinement, and Optuna TPE [1] strategies); and (iv) *agents-as-a-judge* [47, 46]: an LLM judge that selects among candidate behaviors and grades trajectories against written rubrics.

A review of the world-model landscape reveals distinct strategy families with a recurring tension—fidelity and scale on one side; verifiability, sample efficiency, and steerability on the other (Table 1).

Contributions. Organized around the world-time-compute thesis above:

1. **World-time compute (the finding).** A verified world model is *training signal*: fine-tuning an LLM on trajectories traversed through many world models of a domain lifts its generalization to *held-out* worlds it never trained on—a training-time analogue of test-time compute. The gain is largest where capability is scarcest (+29 points at 0.5B, narrowing to +3 at 32B as the task saturates) and, on a coding-world family, transfers at best-of-k to

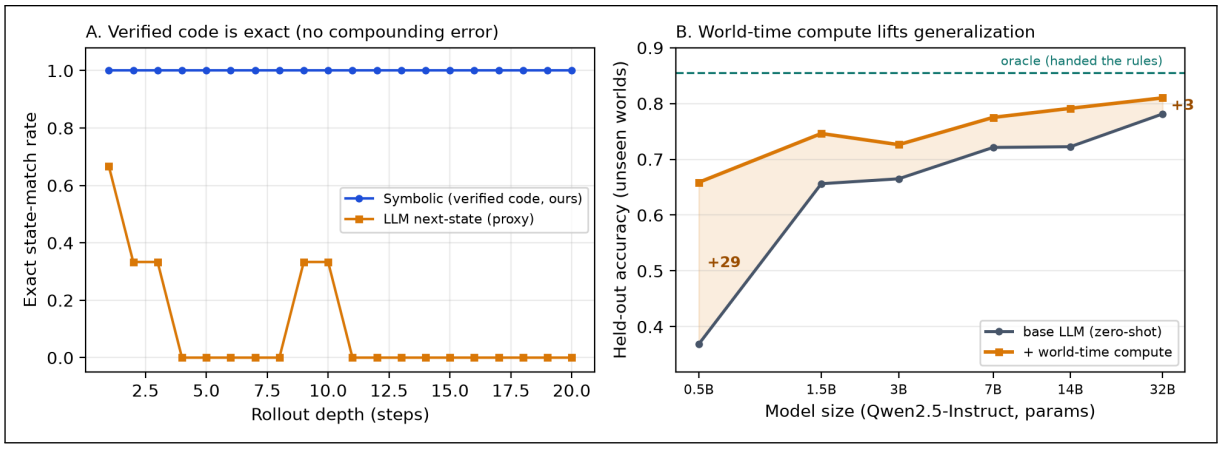

Figure 1: **The paper in one figure.** *A (foundation):* On a ground-truth-instrumented world, dynamics synthesized and verified once as code (blue) match the oracle exactly at every depth of a 20-step rollout, while per-step LLM next-state prediction (orange)—a structural stand-in for learned neural dynamics—first diverges at step 2.3 on average and never completes an exact rollout. Programs encode rules, not statistics, so the compounding-error failure mode of learned world models is eliminated by construction, at zero training cost. *B (payoff):* what that exactness unlocks—*world-time compute*. Fine-tuning an LLM on trajectories traversed through 60 verified *train* world models lifts held-out diagnostic accuracy on 20 *unseen* worlds toward the rules-given oracle, with the largest gain for the smallest models (+29 points at 0.5B, shrinking to +3 at 32B)—a generalization lever that is strongest where capability is scarcest (E74, §4.3).

HumanEval (87%→91%) while not improving greedy pass@1. We argue *exact* labels—not task variety alone—are what separates this from domain randomization, and identify the control that isolates it.

2. **Why it must be verified code (the foundation).** Traversal is only safe if the worlds do not lie. On three ground-truth-instrumented worlds, verified synthesized dynamics are exact on 24/24 20-step rollouts (95% CI [0.86, 1.00]) and answer 10× out-of-distribution probes at 100% from *zero* transitions, whereas the LLM next-state proxy completes 0/24 and a 10,000-transition MLP scores 0% out of distribution—so a traversed world distills rules, not a hallucinated approximation. Where dynamics are writable, planning through the verified model matches ground truth and beats planning through a hallucinating one; on MiniGrid and cartpole it solves 0-shot what a trained DreamerV3 and a frozen-perceptual V-JEPA-2 need 10^4–10^5 interactions to learn—because, handed the dynamics, it need not learn them (a scope property, not a contest).

3. **The framework that makes it cheap.** OpenWorld, a zero-dependency Python framework that turns authoring a verified code world model into a minutes-scale, push-button activity: a local LLM (or Claude Code) declares the spec—symbolic state, a verified `transition`, perceptors (the perceive boundary), composition (worlds within worlds), dial-steerable objectives, an emit/act boundary—which is checked, served from one command, and shared as a portable artifact. This is what makes assembling the *many* worlds world-time compute needs practical.

4. **A prototyping benchmark for many worlds.** A 100-world benchmark across 6 regulated sectors (healthcare, financial services, legal, cybersecurity, energy, agentic-AI) in which Claude Code built every world at a median of 2.8 minutes (p90 3.9, max 6.1): 100% pass a structural `validate_spec` gate (well-formed, round-tripping, servable) and 99% additionally clear a

behavioral audit (load, random-rollout, state changes). These gates establish *servability*, not fidelity to intent; all 100 ship as runnable recipes (companion framework paper [35]).

5. **Benchmarks, protocol, and reproducibility.** Three ground-truth-instrumented worlds, a 20-task program-repair benchmark (the coding world), and 78 seed-fixed experiments with paired statistical tests, all regenerable from one repository.

6. **Measured guardrails (verification).** Verification is evaluated, not assumed: ablations isolate what each gate buys (+0.24 absolute probe accuracy over blind acceptance) and decompose the mechanism (the filter holds quality, the repair loop converts 62% acceptance into 100%). Crucially, verification certifies executable, invariant-respecting code, not rule-correctness: on the weak 3B generator the full gate accepts every program yet *none* is branch-exact (a 100% branch-level false-accept rate), so exactness reflects generator capability gated by cheap checks, not the gate.

Positioning in the field. OpenWorld builds on the *programmatic* code-world-model program [40, 11, 32, 18]—LLM-synthesized, verified code dynamics, as distinct from *neural* code world models that learn dynamics in weights [23] and from LLM synthesis of symbolic world models via test-time scaling [43]—but its contribution is what that machinery *enables*: world-time compute, which turns many verified worlds into exactly-labeled training signal. Its closest relatives are the *self-training* methods that retrain a model on its own verifier-filtered outputs (STaR [45], ReST [13, 36], expert iteration [4], rejection-sampling fine-tuning [44], verifiable rewards [17]); we differ in *what* verifies (a full world model with dynamics, so the unit is a verified trajectory, not a checked answer) and in the *generalization axis* (held-out worlds in a family). It is also distinct from domain randomization and procedural task generation [41, 10, 29], where experience comes from a single (often hand-coded or learned) simulator: here every world is independently synthesized and verified, and the open question we frame is whether that label *exactness*, not task variety, drives the held-out gains. The supporting machinery extends the open-specification stance of the value-alignment literature [38, 21] and judge-based behavior selection [47]. We benchmark on consumer hardware with 7B/3B local models, the regime where learned world models are least accessible and training-free approaches matter most.

2 Problem Setting and Positioning

We target symbolic-state environments: worlds whose state is a JSON-serializable structure, whose action set is discrete, and whose dynamics follow declarable rules. This covers operational simulations (triage, negotiation, portfolios, sprints) and program repair, but not pixel-native domains; we return to this boundary in the limitations. The practitioner declares a world (state schema, actions, plain-language rules), and the framework must produce a faithful, fast, auditable simulator plus the machinery to steer and optimize behavior within it. The adversary here is not a security attacker but *model error itself*: hallucinated transitions, silently wrong code, reward misspecification, and value drift.

A world model, formally. An OpenWorld world model is a tuple

$$W \;=\; \big(\mathcal{S},\ \mathcal{A},\ s_0,\ \tau,\ \{(o_i, w_i)\}_i,\ \mathcal{I}\big),$$

where $\mathcal{S}$ is a space of *symbolic states* (each $s \in \mathcal{S}$ a JSON-serializable map of named fields to values), $\mathcal{A}$ is a finite, discrete action set, $s_0 \in \mathcal{S}$ is a distinguished initial state, and the dynamics

$\tau : \mathcal{S} \times \mathcal{A} \to \mathcal{S}$ is a *deterministic program* returning the successor state (an action illegal in a state acts as a no-op, so τ is total). A stochastic world threads an explicit pseudo-random seed through the state, so τ stays a pure function and every rollout is replayable; OpenWorld thus models stochasticity as replayable seeded determinism rather than a transition kernel $\mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$. Objectives $o_i : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ score a transition (s, a, s'), are combined as $\sum_i w_i o_i$, and are weighted by *dials* w_i (fixed or tunable); invariants $\mathcal{I}$ are predicates $\iota : \mathcal{S} \to \{\top, \bot\}$ required to hold on every state reachable from s_0 under τ. The practitioner declares $(s_0, \mathcal{A}, R, \{(o_i, w_i)\}_i, \mathcal{I})$—an initial state, the actions, natural-language rules R, objectives with their dials, and invariants; the schema of s_0 induces $\mathcal{S}$, and the framework must *produce* τ from R.

It produces τ by *synthesis under verification*, not by learning. An LLM proposes a program $\hat{\tau}$ from R; $\hat{\tau}$ is accepted only if it (i) parses with the required signature, (ii) executes in a sandbox over a probe set $P \subseteq \mathcal{S} \times \mathcal{A}$ (in our implementation, each action applied at s_0) and satisfies every $\iota \in \mathcal{I}$ on the resulting states, and optionally (iii) survives a second-model critique against R. Verification therefore certifies signature-conformance and invariant-consistency on P; it does *not* certify $\hat{\tau} = \tau^\star$ against a ground truth—it bounds and surfaces error rather than eliminating it. The accepted $\hat{\tau}$ is then *fixed*: a depth-t rollout $s_t = \hat{\tau}(\cdot, a_t) \circ \cdots \circ \hat{\tau}(\cdot, a_1)(s_0)$ is computed by executing code, so *if* $\hat{\tau} = \tau^\star$ on the reachable set, it is exact at *every* depth, bit-exactly. Contrast a learned or per-step-predicted transition $\tilde{\tau}$ resampled each step: writing $\epsilon = \Pr_{(s,a)\sim\pi}[\tilde{\tau}(s, a) \neq \tau^\star(s, a)]$ for its single-step error rate over states visited under a policy π, the probability that a full t-step rollout is exact is $\prod_{k<t}(1 - \epsilon_k) \approx (1 - \epsilon)^t$ under independent, constant-ϵ steps—and strictly lower once an early error pushes the rollout off-distribution, where ϵ grows. A symbolic world fixes its error *once*, at synthesis time, where it is systematic and auditable; a learned world risks a fresh error every step, where it compounds. This tuple, and that property, are what the rest of the paper measures.

Table 1 situates the approach against the principal world-model strategy families surveyed in the accompanying literature review.

System	Approach	Gap OpenWorld targets
World Models [15]	VAE + MDN-RNN latent dynamics	millions of env. steps; opaque latents
DreamerV3 [16]	RSSM, discrete latents, imagination	compounding rollout error; values in weights
MuZero [34]	value-equivalent latent planning	ungrounded states; incomplete context
IRIS / Δ-IRIS [24, 25]	VQ tokens + autoregressive transformer	token budgets; drift over horizon
DIAMOND [3]	diffusion dynamics in pixel space	GPU-heavy; no symbolic audit
Genie [5]	generative interactive video (11B)	frontier-scale compute; closed
Sora / World Labs [31, 12]	generative video / 3D "spatial intelligence"	perceptual, not symbolic; no auditable state
V-JEPA [22]	joint-embedding video prediction in latent space	perceptual; latent, unauditable; trained
NE-Dreamer [26]	decoder-free next-embedding prediction	still trained; latent, unauditable
Meta CWM [23]	*neural* code world model (32B, dynamics learned in weights)	not auditable code; frontier-scale training
IVML [43]	LLM-synthesized symbolic world models via test-time scaling	per-task synthesis; not a reusable substrate
WorldCoder / GIF-MCTS [40, 11]	LLM-synthesized code dynamics	bespoke pipelines, no reusable framework
PoE-World [32]	products of programmatic experts	per-domain expert engineering
NeSyS [27]	symbolic rules constrain neural logits	still requires fine-tuning data
OpenWorld (this work)	declared worlds; verified code dynamics; moral dials; tuner; judge	—

Table 1: Positioning against representative world-model strategies from the learned and symbolic families.

3 Experimental Setup

Worlds with ground truth. Three deterministic oracle worlds instrument every dynamics comparison: *sprint* (backlog/debt/bug dynamics with an integer-division interaction), *orchard* (resource pool with per-agent tallies), and *triage* (queues, a clock, and deterioration events). Oracles are hand-written `FunctionTransitions`; synthesized and learned-style engines are compared against them exactly, including on a fixed probe suite that exercises clamping, empty-queue, and interaction branches.

The coding world. Program repair is the flagship use case: well-known, objectively scored, and natively symbolic [6]. Each of 20 tasks is a buggy Python function plus a hidden test suite spanning classic defect archetypes (off-by-one ranges and binary-search bounds, inverted comparisons, missing normalization, wrong base cases, unsorted medians, order-destroying dedup, early imbalance, whitespace splitting, accumulator resets, integer-division truncation, overlapping counts, dropped

final runs, and degenerate-input handling). The world's transition *is* test execution: a submitted patch runs bit-exactly in a sandbox with a wall-clock alarm (a looping patch fails rather than hangs), and failing-test feedback enters the state for the next attempt.

Benchmark-scale repair (E28–E29). Beyond the single-function benchmark, two SWE-bench-style suites probe repair at module scale. Each instance carries a natural-language issue report (symptoms and a repro, never the fix), a buggy module (functions or a stateful class), two hidden suites (`fail_to_pass` exercising the defect, `pass_to_pass` guarding regressions), and an explicit world specification whose `submit_patch` dynamics run both suites bit-exactly; solving requires zero failures in *both*, so a fix that breaks a passing test does not count. The *atomic* suite (E28, $n = 6$) plants one issue-described defect per instance; the *staged* suite (E29, $n = 15$) plants a second, latent defect that surfaces only as a new failing test once the issue-visible defect is fixed. Both run a paired ablation across the `qwen2.5` ladder (1.5B/3B/7B): the same model *single-shot* (one completion from issue + module) versus *in-world* (up to 4 `submit_patch` steps; the first prompt is identical to single-shot, and exact failing-test feedback enters from the second attempt onward). An expanded 20-instance atomic suite with additional regression traps ships in the repository (`datasets/openworld-repairbench/`).

Composition, nesting, and changing rules (E30–E32). Three experiments exercise `CompositeWorld`, a wrapper whose state nests its children's states under namespace keys. Children run *unmodified*: the composite slices a child's namespace out, steps the child's own transition, and writes it back. Coupling is explicit—sideways through `Bridge`s (an ordinary transition over the two-slot dict of the coupled children's states, so the same synthesis-and-verification pipeline applies to bridges unchanged), upward through `Aggregator`s (derived from leaves, never simulated), and agents travel between children over `Route`s whose crossing effects (tolls, vetoes) are themselves verifiable transitions. `PhasedTransition` handles rules that change over time: ordered (trigger, transition) phases with sequential, irreversible advance, every phase constructed and verified *before* the run. E30 holds one system fixed—16 internal rules plus four ring couplings—and varies only how synthesis is posed: one monolithic prompt over a flat namespaced state versus eight small prompts (four 4-rule sectors, four one-rule bridges), each verified independently and assembled into a `CompositeWorld`; both conditions score on the same ground-truth probe suite (7B generator, three replicates each). E31 replays a 20-step script (leaf actions, ticks, and two travel attempts across a toll route) on a three-level composite against an independent flat-dict oracle that imports nothing from the composition machinery, checking leaf exactness, aggregator consistency, money conservation, and agent-registry agreement at every step. E32 declares a market whose policy changes at a fixed step mid-rollout and compares phased synthesis (each phase from its own rule text alone), monolithic synthesis from the combined rules-with-change text, and the LLM next-state proxy, on closed-loop rollouts crossing the boundary.

Models and hardware. All experiments use local models served by Ollama on an Apple-silicon laptop: `qwen2.5:7b` (generator, judge, critic in E3), `qwen2.5:3b` (small generator in E2/E3), `qwen2.5:1.5b` (repair agent in E5/E6/E13/E17), and—for cross-family replication (E16)—`llama3.1:8b` (Meta) and `gemma2:9b` (Google). The trained baselines in E12/E19 are plain numpy models. No cloud APIs, no fine-tuning, and no training compute beyond the baselines' seconds-scale fitting.

Baselines. The learned-dynamics family is represented two ways. (i) `LLMTransition`: the same backbone predicting each next state directly from the same description and rules—sharing the

symbolic engine's knowledge while exhibiting the per-step stochastic prediction structure of learned models. (ii) *Trained* models (E12): a two-hidden-layer MLP and a 1-nearest-neighbor memorizer, each trained on $K \in \{100, 1000, 10000\}$ environment transitions collected by a random policy—real learned dynamics with a measured sample-efficiency curve. We are explicit that neither is a trained Dreamer; laptop-scale reimplementation of the pixel-native family is infeasible, which is itself part of the comparison (DreamerV3-class systems require millions of environment steps [16]).

Metrics and statistics. Rate metrics carry 95% Wilson confidence intervals. Dynamics fidelity uses exact-state match (every field equal), first-divergence step, and final-state L^1 error. Program repair reports pass@1 and pass@budget (4 attempts); the controlled selection comparison (E13) is a paired design on shared candidate sets with an exact two-sided McNemar test. Judge alignment uses Spearman rank correlation with permutation p-values (10,000 shuffles). Seeds are fixed in each script; every number in this paper regenerates via `python3 scripts/make_paper_assets.py`.

Calibration, pilots, and internal review. Design changes made after pilots are reported rather than hidden: (a) the verifier ablation (E3) uses the 3B generator at high temperature because the 7B generator's first attempts passed every gate, leaving nothing for conditions to discriminate; (b) the repair agent in E5/E6/E13 is the 1.5B model because both the 7B and 3B agents saturated the original benchmark (pass@1 = 100%)—with failing-test feedback in the state, these archetypes are easy for capable models. The capability ladder (1.5B: 70%, 3B and 7B: 100% on the original suite) is itself a finding. All LLM-dependent numbers are specific to the stated Ollama snapshots and Q4_K_M quantization.

4 Results

4.1 Head-to-head: symbolic vs. learned-style dynamics

This section measures *dynamics fidelity*—predicting a world's next state from the current state and an action—not the program-repair task of the coding world. The practitioner *declares* each world's rules; the question is how to turn declared rules into a faithful simulator. The symbolic engine compiles the rules to a verified `transition` program once and then executes it; the learned-style engine predicts each next state directly. The sprint world's `bugs += debt//4` coupling is an arbitrary *declared* rule, not a natural law to be guessed: the point is not that a model should anticipate it, but that once it is declared, compiling it to code runs it exactly, whereas predicting it step by step compounds error.

Table 2 and Figure 1 present the primary comparison (E1, E4, E10), and Table 3 replicates the protocol across all three instrumented worlds with eight action scripts each (E11). The verified synthesized engines are exact on 24/24 rollouts (95% CI [0.86, 1.00]) against 0/24 (CI [0.00, 0.14]) for per-step LLM prediction, and—because planning executes Python rather than a forward pass—roll out at 14,997 steps/s against the LLM engine's 0.32 steps/s, after a one-time synthesis cost of a few seconds.

4.2 The central result

Verified code synthesis eliminates compounding rollout error at zero training cost. The mechanism is structural, not statistical. A learned (or per-step predicted) transition is sampled each step, so per-step error ϵ compounds roughly as $1 - (1 - \epsilon)^t$: with the measured single-transition error of the LLM engine (60% of probes wrong), divergence by step 2.3 is exactly what the arithmetic

Engine	First divergence (step)	Final L1 error	OOD exact (10×)	Steps/s
Symbolic (verified code, ours)	21.0	0.0	100%	14,997
LLM next-state (proxy)	2.3	10.3	40%	0

Table 2: Head-to-head engine comparison on the sprint world (E1, E4, E10): mean first-divergence step and final L^1 error over three 20-step rollouts against a ground-truth oracle; exact transition accuracy on ten 10× out-of-distribution probes; and rollout throughput. A first divergence of 21 means no divergence occurred within the rollout.

World	Code: exact rollouts	LLM: exact rollouts	LLM first divergence
orchard	8/8	0/8	11.2
sprint	8/8	0/8	1.4
triage	8/8	0/8	2.0
Total	**24/24** [0.86, 1.00]	0/24 [0.00, 0.14]	—

Table 3: Multi-world replication (E11): exact 20-step rollouts per world, eight scripts each, dynamics synthesized once per world by the 7B generator. Totals carry 95% Wilson CIs.

predicts, and twenty-step plans are fiction (final L^1 error 10.3). The symbolic engine pays its entire error budget *once*, at synthesis time, where it is auditable: if the accepted program is right, every rollout of any depth is right, bit-exactly. This is the CWM hypothesis [40, 18] made measurable on consumer hardware.

The single-number divergence 2.3 is from one world (sprint, n=3 rollouts, one seed) and is *not* representative: across the three worlds (E11, 24 rollouts) the LLM proxy's first divergence ranges from step ≈1.4 (sprint) and 2.0 (triage) to ≈11 (orchard), because compounding depends on the per-step error rate, which varies by world. The robust, world-independent claim is the exact-match contrast, not the mean divergence step: code engines complete 24/24 of 24 twenty-step rollouts bit-exactly (95% CI [0.86, 1.00]) versus 0/24 for the LLM proxy. We treat the divergence figure as illustrative of the mechanism and the exact-match rate as the result.

4.3 World-time compute: traversing world models to generalize from fewer examples

The results so far buy a *correct* world model cheaply. A verified world model is also a *generator*: it produces unlimited, exactly-labelled experience and exposes its own rules, so an agent can *traverse* many worlds of a domain and distil the shared skill. We call spending compute this way—fine-tuning a model on trajectories drawn from traversed, verified world models rather than on scarce real examples—**world-time compute**, a training-time analogue of test-time compute [37, 30, 14] (and, when spent per world at inference, of test-time training [2]).

We test it on a *family* of diagnosis worlds (E74): one POMDP template (a hidden disease; order tests to reveal symptoms; commit a diagnosis) instantiated as 60+20 specialties that share a goal and action grammar but differ in their symptom → disease structure. We hold out whole specialties (a world-level train/test split, as in supervised learning): fine-tune a model on traversed *train* specialties, then measure diagnostic accuracy on 20 *unseen* specialties (800 cases), bracketed by a prior-only floor (34%) and an oracle handed the rules (86%). Figure 2 walks through a single such world—a hidden-state POMDP the model learns to invert. To be precise about what transfers: each test prompt *states that specialty's profiles* (its rules) in context, and the base baseline is conditioned on the *same* prompt—so the lift is not the fine-tuned model reading a table the baseline cannot

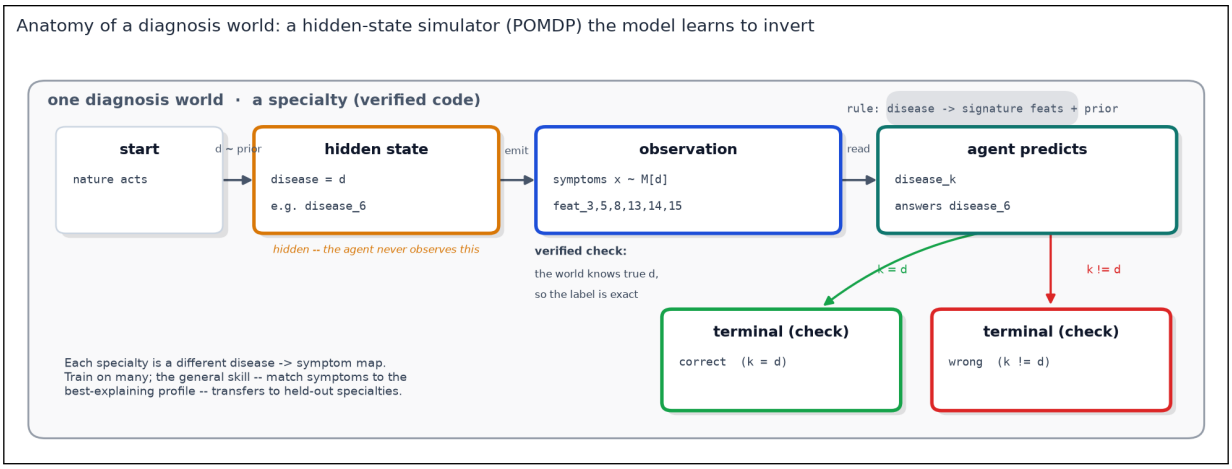

Figure 2: **Anatomy of one diagnosis world (E74–E79).** Each *specialty* is a small hidden-state simulator: nature draws a hidden *disease* from a prior, the world emits *symptoms* from that disease's signature features (plus background noise), and the agent must invert the observation to name the disease. Because the world is *verified code*, the true disease is known, so the training label is *exact*—a traversed world distils the right rule, not a hallucinated one. Each specialty is a different symptom → disease map; training on many and testing on *held-out* specialties forces the general skill (match symptoms to the best-explaining profile) rather than memorisation.

(both see the rules), but fine-tuning teaching the model to *apply* provided rules in a way that carries to symptom → disease maps it never trained on. This is the in-context regime of test-time training, one rung more general (across rule-sets, not within one); internalising fully *latent* dynamics is harder and we do not claim it here.

Two findings (Figure 3). *(i) Generalization, not memorization:* world-time compute lifts held-out accuracy at every model size—from 37% to 66% at 0.5B, and from 78% to 81% at 32B—moving the model toward the rules-given oracle on specialties it never trained on. *(ii) It is a small-model multiplier:* the gain is largest where capability is scarcest (+29 points at 0.5B, bootstrap CI over held-out specialties well clear of zero) and *shrinks* as larger models approach the rules-given ceiling—by 32B the lift is only +3 points and its CI includes zero, because the task is nearly saturated (oracle 86%). Whether the gain re-emerges once capable models are given headroom—a deliberately harder world family—is a clean falsifiable follow-up we are running. An offline analogue confirms the distilled structure is a sample-efficiency win regardless: a learner that has seen the family diagnoses a held-out specialty from fewer cases than one learning from scratch (70% vs 68% at a single case per disease).

The *family* is what makes this work. Fine-tuning on *heterogeneous*, unrelated worlds that share no task structure yields no such transfer (E71, E73): there is no shared skill to distil, so a policy expert on one world is no better than chance on another. World-time compute pays off precisely when the traversed worlds form a domain—then more traversal becomes a lever on generalization, complementary to train-time data and test-time search. (Scope: the model-size sweep mixes precisions—0.5B–7B are bf16 LoRA, the larger sizes 4-bit QLoRA—and the per-*number*-of-worlds axis saturates quickly, so we claim a compute *lever*, not a clean world-count scaling law.)

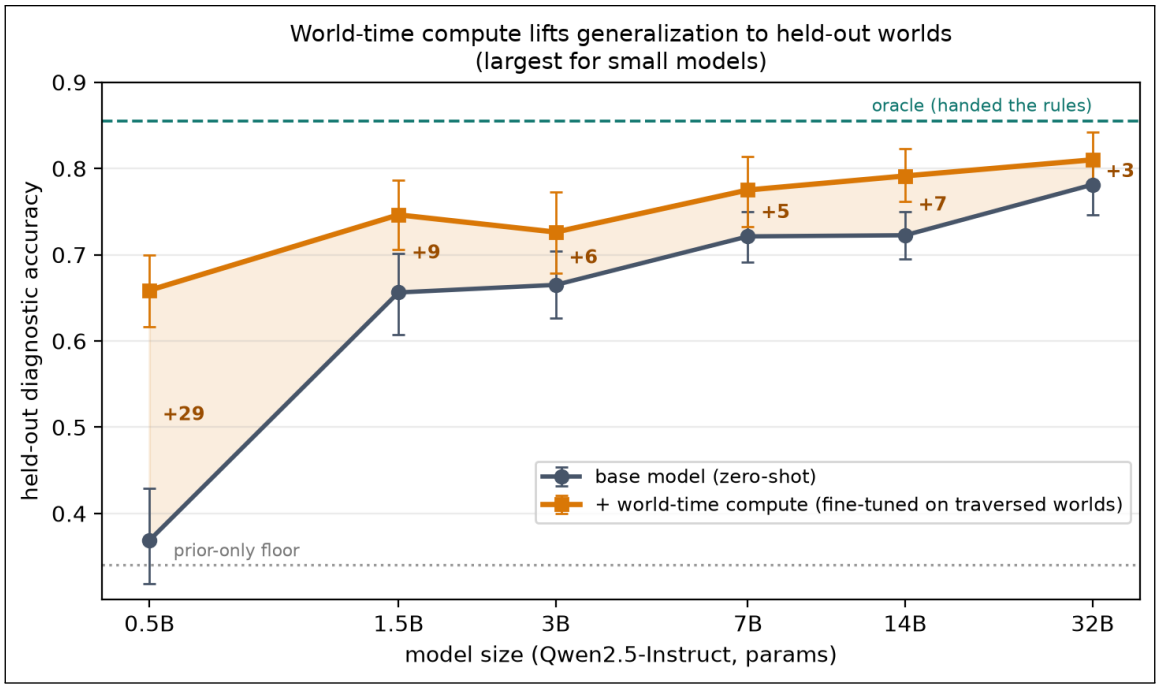

Figure 3: **World-time compute (E74).** Held-out diagnostic accuracy on 20 *unseen* diagnosis specialties, before (gray) and after (orange) fine-tuning on trajectories traversed through 60 *train* specialties' verified world models, across model size. The shaded band and labels are the per-size generalization gain. Fine-tuning moves every model toward the rules-given oracle, with the largest lift for the smallest models—world traversal is a sample-efficiency multiplier, strongest where capability is scarcest. Fine-tuning on unrelated, heterogeneous worlds gives no such transfer (E71, E73).

4.4 Scaling the lever, and a coding-world transfer test

Two follow-ups sharpen the principle: does it *scale* with the number of worlds, and does it hold on a *different* use case authored as real verified-code worlds?

World-count scaling (E76). We fix the model (7B) and the hard family and vary only the number of train specialties traversed, holding the test specialties fixed (Figure 5). Too few worlds *hurt* (the fine-tune overfits a narrow slice, below the 48% base); the gain then rises monotonically and is *still climbing at 512 worlds* (59%, +10 points), heading toward the 69% oracle—no plateau. So world-time compute is a genuine scaling lever in the number of worlds, when the task has headroom; the offline empirical-Bayes prior saturated at ≈2 specialties precisely because it has nothing of the world's *skill* to keep learning.

Exact labels are the lever (E78b). Scaling shows *more* worlds help; this ablation shows *why verified* worlds are needed. Holding the world count fixed (256 train specialties) and the patients, prompts, and example count identical, we vary only the *training labels*: a fraction p of each world's diagnoses is replaced by a confusable disease drawn from the posterior over the others—the structured error a non-verified or learned world model makes when its rules are slightly wrong. The held-out test labels stay exact. Held-out accuracy declines monotonically with corruption, from 35% at p=0 (verified) down to 19% at p=1—a fully-wrong world yields *no* generalization gain (back to the no-fine-tune base), with a smooth penalty in between (Figure 4). So it is label *exactness*, not task variety, that drives the effect—the property that separates world-time compute

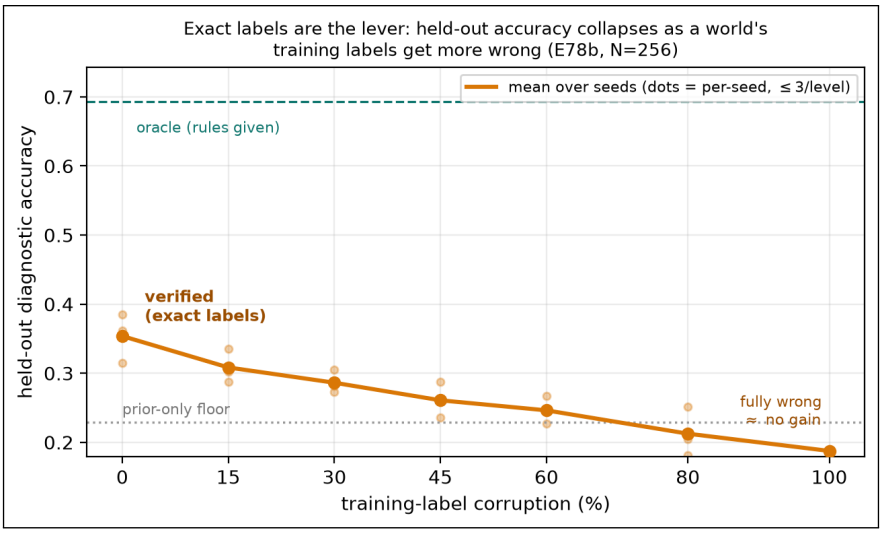

Figure 4: **Exact labels are the lever (E78b).** Held-out diagnostic accuracy after fine-tuning on 256 worlds whose *training* labels are corrupted at rate p (same worlds, patients, and prompts throughout; the test labels are always exact). Verified worlds (p=0) sit at the top; accuracy falls monotonically to the no-fine-tune base as the world becomes fully wrong (p=1). Label exactness—not task variety—is what world-time compute requires. Dots are per-seed values (3 seeds/level).

Model	Synthetic p@1	Synthetic p@5	HumanEval p@1	HumanEval p@5
0.5B	0.17→0.27	0.38→0.55	–→–	–→–
1.5B	0.51→0.55	0.78→0.84	–→–	–→–
3B	0.78→0.70	0.85→0.91	–→–	–→–
7B	0.73→0.80	0.84→0.95	0.78→0.70	0.87→0.91

Table 4: **Coding world family (E77).** Sampled pass@1/pass@5 (n=5), base → world-time-compute (fine-tuned on 164 verified coding worlds), on synthetic held-out tasks (all sizes) and HumanEval (7B). World-time compute helps in-domain best-of-k at every size; on HumanEval it hurts greedy pass@1 but transfers positively at pass@5.

from domain randomization, and the reason the worlds must be *verified* code (3 seeds per level; per-seed values shown in Figure 4).

A coding world family (E77). Coding is the cleanest verified-code world we have: the transition is execution and the oracle is the tests passing. We had a model author 164 function-implementation tasks, each admitted only after its reference solution passes its own sandboxed `pytest` oracle, and fine-tuned on the traversed worlds (Table 4). *In-domain*, world-time compute lifts best-of-k at every model size (7B pass@5 84%→95%). On *HumanEval*—a real, out-of-distribution benchmark—it *hurts* greedy pass@1 (7B 78%→70%, the synthetic fine-tune narrows the single best guess) yet *transfers positively at pass@5* (87%→91%) from only 164 worlds. That is consistent with E76: 164 worlds is below the count where transfer becomes strong, so scaling the world family—not abandoning the approach—is the path to real-benchmark gains.

4.5 Real data: world-time compute on ARC-AGI (E80)

Every world-time-compute result so far traverses worlds *we* authored (diagnosis, coding), which invites the sharpest objection: the gains might live in our generator, not in the world models. We

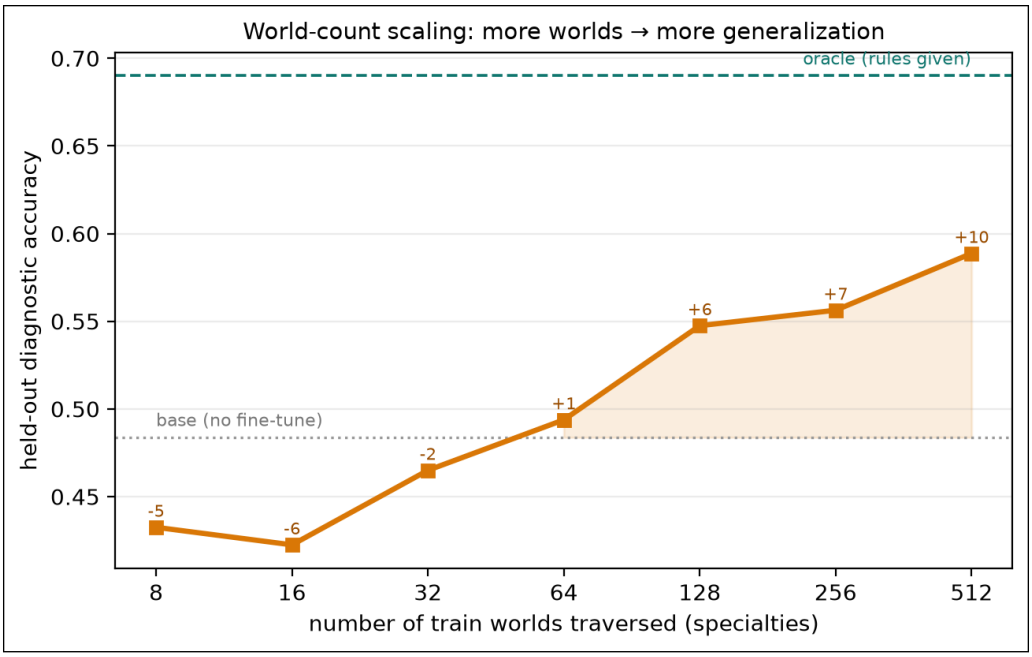

Figure 5: **World-count scaling (E76).** Held-out diagnostic accuracy of a 7B model vs the number of train specialties it traversed (hard family; test specialties fixed). Labels are the gain over the no-fine-tune base. Few worlds hurt; the gain rises monotonically and has not plateaued at 512, climbing toward the rules-given oracle. *More worlds → more generalization.*

therefore port the mechanism to a benchmark we did not write—**ARC-AGI** [7, 8], the human-authored abstraction-and-reasoning corpus (400 training + 100 evaluation tasks; we use ARC-AGI-1, with ARC-AGI-2 [9] a harder successor). It is an unusually clean fit: each task *is* a world—a hidden grid-transformation rule shown only through a few input → output demonstrations—so unlike our synthetic worlds (where the rule is *stated* in the prompt) the rule here is *latent* and must be induced. Labels are exact (a predicted output grid is right or it is not, no judge), and the train and evaluation tasks are disjoint with a novel rule each, so "generalize to held-out worlds" *is* the ARC challenge. We expand each task into many world variants with rule-preserving augmentation (the dihedral group × colour permutations) and run two tests on a 4-bit Qwen2.5-7B. The per-task test-time-training recipe—a fresh LoRA fit on a task's augmented demonstrations—follows Akyürek et al. [2], who established it as the effective approach on ARC (cf. induction/transduction program search [19]); our contribution is not the recipe but to read it as *world-time compute on a verified world* and to add the corrupted-label ablation that isolates whether the *exactness* of the labels—not fine-tuning per se—drives the gain. Figure 6 diagrams the regime end to end—world model → augmented, exactly-labelled rows → QLoRA fine-tune (frozen 4-bit base, trainable LoRA), with the two strict train/test splits—and Figure 7 reports the results.

(i) Cross-task transfer is weak. Training one LoRA on N real training worlds and evaluating zero-shot on disjoint held-out worlds barely moves exact-match (3% base → 5% at 200 worlds). This is the expected, honest boundary: every ARC rule is novel by construction, so a single shared policy has little to transfer—consistent with E76/E77, where transfer turns strong only well above the world counts we can afford here.

(ii) Test-time training—world-time compute spent per world—works. For each held-out world we fit a fresh LoRA on *that task's own demonstrations* (its test pair strictly held out), then predict its test grid. This is the purest form of the thesis: compute spent learning a verified world yields

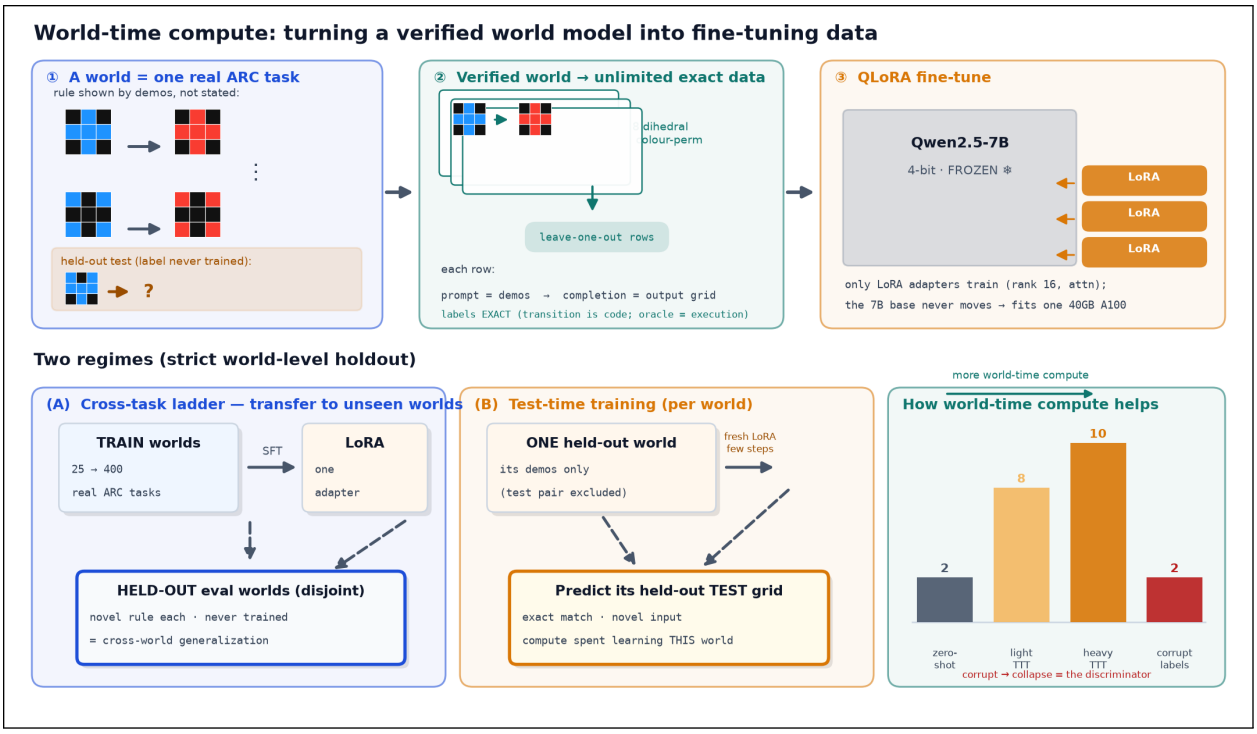

Figure 6: **How world-time compute fine-tunes on a verified world model (E80, ARC-AGI).** *Top:* each real ARC task is a world whose rule is shown by demonstrations, not stated; rule-preserving augmentation (dihedral × colour) turns it into unlimited *exactly*-labelled rows (the transition is code, so the oracle is execution), which LoRA-SFT a frozen 4-bit Qwen2.5-7B—only the small adapters train. *Bottom:* two strict world-level holdouts—(A) the cross-task ladder trains on 25–400 worlds and is scored on *disjoint* held-out worlds; (B) test-time training fits a fresh adapter on a single held-out world's demos (its test pair excluded) and predicts that test. The payoff bars track exact-match vs. world-time compute, with a corrupted-label arm as the discriminator: if the lift survives label corruption it is not the verified labels—not the optimizer—doing the work.

generalization within it. Exact-match climbs from 2% zero-shot to 8% (light) and 10% (heavy)—+8 points as we spend more world-time compute. The decisive mechanism check is the **corrupted-demonstration ablation**: randomizing the demo outputs (same grid structure, wrong labels) erases the lift if the model is exploiting the *exact* labels rather than benefiting from fine-tuning per se. It does—the corrupt arm falls to 2% (Figure 7), at or below the 2% zero-shot floor, establishing that the lift requires *verified* labels: the model learns each world's real rule, not grid statistics.

These are mechanism numbers, not a leaderboard entry: a single 7B at 4-bit with one seed over 40 held-out worlds, far below ARC solvers that add deep augmentation, voting, and program search. The claim is narrow and, we think, the important one: world-time compute—spending compute to learn verified world models—lifts generalization on *real, human-authored* tasks, exactly where the "you generated it" critique bites hardest, with the corrupted-label ablation as the test of whether that lift is owed to the labels' *exactness*.

4.6 Across real domains and modalities—and where it stops (E80)

ARC is not a one-off. We ran the *identical* per-world test-time-training protocol on two further real benchmarks from different fields, plus one different *modality* (Figure 8): **List Functions** [39] (60 hidden list→list functions induced from input/output pairs), **CLRS-Text** [20, 42] (29 classic

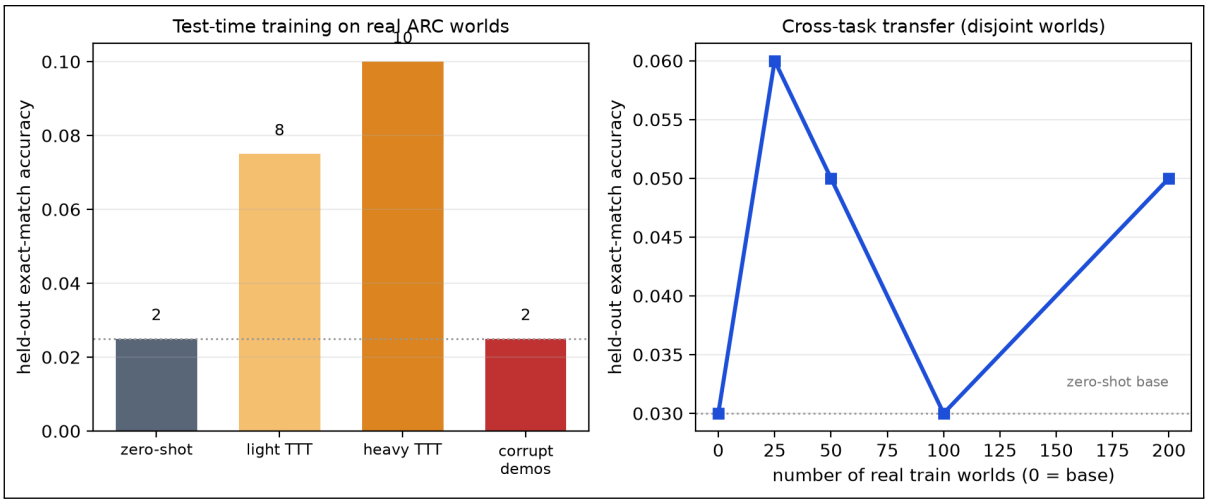

Figure 7: **World-time compute on real ARC-AGI (E80).** *Left:* per-world test-time training lifts held-out exact-match from zero-shot through light to heavy test-time compute; a corrupted-demonstration arm (wrong labels, same structure) shows verified labels are necessary—the lift vanishes under corruption (2%). *Right:* cross-task transfer (train on N real worlds, evaluate on disjoint held-out worlds) is weak, as expected from ARC's per-task novelty. Single 4-bit Qwen2.5-7B; mechanism, not leaderboard.

algorithms—sorting, graphs, dynamic programming—each a hidden procedure to execute exactly), and **Bongard-RWR** [28] (221 real-image visual concept-induction problems, via frozen DINOv2 features + a per-world head).

The trend replicates on every symbolic domain. Held-out exact-match *scales* with world-time compute and *collapses* to the floor when the demonstrations' labels are randomised: List Functions 35%→66%→69% (heavy 95% bootstrap CI over worlds [59, 77]), corrupt 3%; CLRS 9%→13%→17%, corrupt 4%; ARC 2%→8%→10% (CI [2, 20]), corrupt 2%. Same mechanism, three independent real benchmarks, and in each the lift is owed to the labels' *exactness*—the discriminator from §5 holds everywhere.

Where it works: few-step reasoning. The *size* of the effect tracks the *depth of reasoning* an instance requires. World-time compute pays off most where the hidden rule is shallow—List Functions, a one-shot list transform, reaches 69%—and least where it demands long multi-step execution—CLRS, whose answers are long algorithm traces, tops out at 17%, the lowest of the three. This is the same boundary as the complexity cliff (E20; mitigated by composition in the companion framework paper [35]): verified and world-time-compute methods excel at tasks solvable in *few reasoning steps* and degrade as the required chain lengthens. It is also exactly where they beat data-hungry learned world models—on the short-horizon **MiniGrid DoorKey** planning task (§4.13) the verified model solves *zero-shot* while DreamerV3 needs 10^4–10^5 interactions and V-JEPA-2 cannot control at all.

The boundary: rich perceptual abstraction. Bongard-RWR is the honest negative: per-world test-time training over frozen visual features stays at chance (base 46%, heavy 48%, corrupt 52%; Figure 8, right)—no scaling, no collapse. When the latent concept must be *induced from rich perception* rather than read from a symbolic input, a small head over frozen features has no purchase. World-time compute is a lever for symbolic, few-step reasoning, not a substitute for

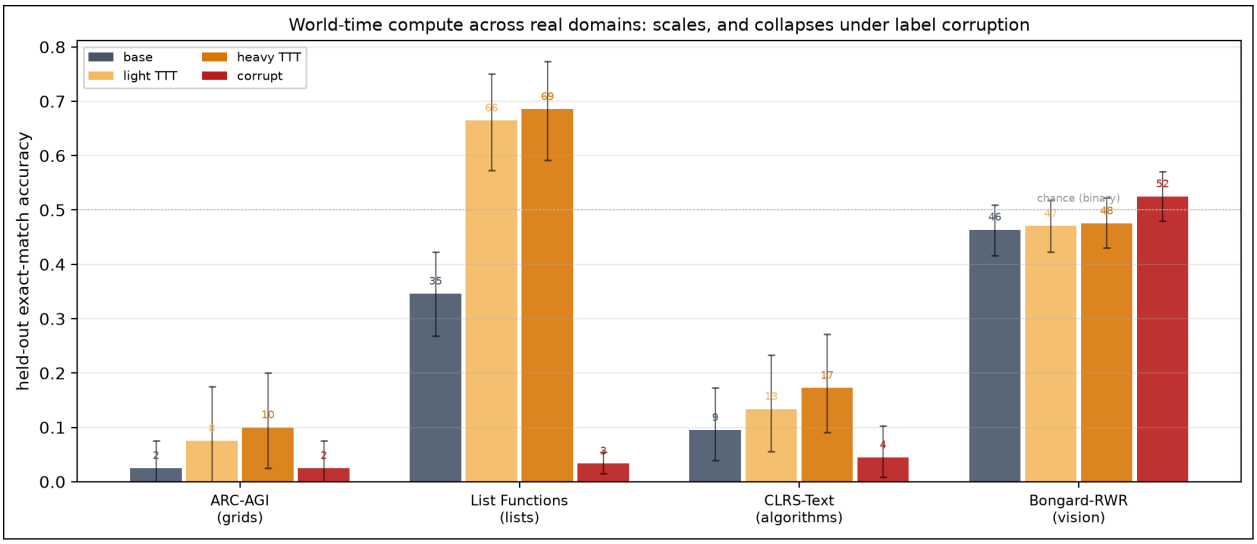

Figure 8: **World-time compute across real domains and modalities (E80).** Per-world test-time training (the same protocol everywhere) on real benchmarks: held-out exact-match for base → light → heavy world-time compute, plus a corrupted-label control; error bars are 95% bootstrap CIs over worlds. The three symbolic domains (ARC grids, List Functions lists, CLRS algorithms) all *scale* with compute and *collapse* under corruption—the lift requires verified labels. Bongard-RWR (vision, right) stays at chance: the boundary where a latent concept must be induced from rich perception, not symbolic input.

perceptual representation learning—the same scope boundary the paper draws throughout.

4.7 Cross-world transfer on a real, human-authored domain (E84)

The real-data results above are *per-world* test-time training—the established recipe [2], not our contribution. The axis OpenWorld is built for is *cross-world* generalization: train once on a family of verified worlds, then generalize to *held-out* worlds. E83 found no such transfer across CLRS algorithms that share no skill; E84 tests the complementary prediction—that a *coherent* family does transfer—on a real, human-authored domain (List Functions). We train one LoRA adapter on 128 disjoint held-in worlds, freeze it, and evaluate 60 held-out worlds it never saw, zero-shot (3 seeds). Three arms share an identical eval: **base** (no adapter), **cross-world** (adapter trained with exact labels), and **corrupt** (the same held-in prompts with randomized targets).

The cross-world adapter reaches 40% (95% CI [35, 45]) on worlds it never trained on, versus 6% ([4, 10]) for the corrupted-label control—a **+34-point** exactness-gated cross-world lift (CI [29, 39], excluding zero), stable across seeds (42%, 43%, 36%; Figure 9). The instruction-tuned base scores 0% by exact match because it answers verbosely (chain-of-thought prose, not a bare list); the corrupt control absorbs exactly that format effect—it learns the output format from the same prompts yet, lacking correct rules, gains only 6%. The gap between cross-world and corrupt is therefore *rule-induction transferring across worlds*, not formatting: an adapter that learned to infer list-function rules from one set of worlds infers *new* rules in worlds it never saw. This is the novel axis—world-time compute as cross-world generalization—demonstrated on a real, human-authored domain, the result E83 left open.

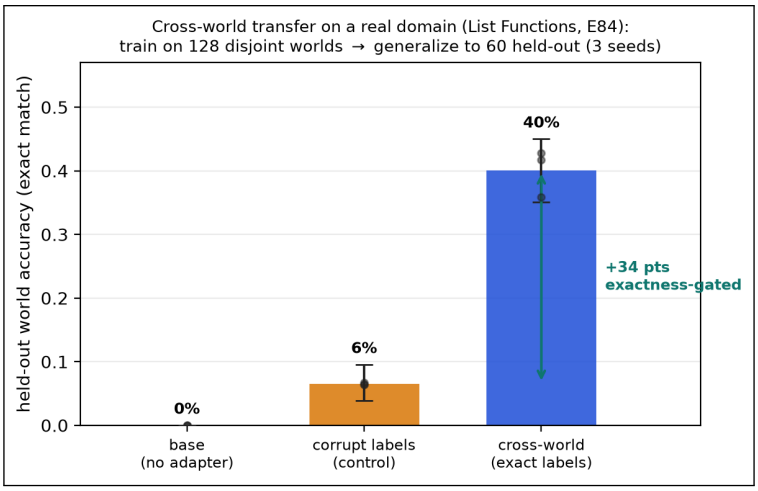

Figure 9: **Cross-world transfer on a real domain (E84).** One adapter trained on 128 *disjoint* List-Functions worlds generalizes to 60 held-out worlds it never saw (40%, 95% CI [35, 45]), while the same training with corrupted labels reaches only 6%—a +34-point exactness-gated lift (CI [29, 39]). This is the cross-world axis (Route 3), on data we did not generate, that E83 (skill-disjoint algorithms) left open.

4.8 A descriptive regularity for world-time compute (E80)

The effect is regular enough across E80 to summarize as a *descriptive regularity*—not a predictive law. It is fit over 5 domains with single-seed runs and a single out-of-sample check, the constants are learner-specific, and the fit is sensitive to domain selection (a sibling fit over a different subset is anti-predictive). We report the *shape* as a compact summary, not a forecast. World-time compute follows a **saturating learning curve**, $\text{acc}(c) = \text{base} + (C - \text{base})(1 - e^{-c/\tau})$, whose asymptote C tracks **identifiability** $\times$ **realizability** $\times$ **headroom** and which collapses to base without **exact labels**. We test it as a conjunction of falsifiable clauses over 333 worlds in 5 domains and three modalities (symbolic text, vision, and tabular); the *form* is what travels, not point predictions.

Clause 1 (scaling): a small probe predicts the asymptote. A light dose of world-time compute predicts the heavy-dose asymptote: within-domain Spearman is 0.98 (ARC), 0.90 (List Functions), 0.73 (CLRS), 0.77 (Bongard, vision), and 0.63 (tabular). A *leave-one-domain-out* fit—train the relation on the other domains, predict the held-out one—reaches R^2=0.51, and the *simplest* model (light alone) beats adding parameters, consistent with a real regularity rather than an overfit. A single out-of-sample check is consistent though not decisive: the fit on grids, lists, algorithms, and vision predicts per-world lift on a brand-new **tabular** domain it never saw, at Spearman 0.60—one held-out modality, not a broad forecast.

Clause 2 (no free lunch): the predictor must be a probe. We tried to predict the asymptote from *zero-training* signals (teacher-forced answer log-probability vs. number of demonstrations). It fails—leave-one-domain-out $R^2 = -0.38$ (anti-predictive). In-context likelihood taps Bayesian updating; the TTT lift taps weight-space learnability, and they diverge. You cannot read the payoff off static signals; you must spend a small dose of the actual process. This negative sharpens the regularity.

Clause 3 (learner-invariance): not just neural networks. The same regularity holds for a *non-neural* learner—an enumerative program synthesizer over a list DSL, with search budget as

the compute axis: held-out exact-match rises and saturates (0.04 to 0.16 over the budget sweep), and corrupting the labels collapses it to 0.00 (no consistent program exists). Only *which* worlds are realizable is learner-specific (the DSL is a different hypothesis class than the network); the curve and the exactness gate are not. The regularity is about the verified-world learning *problem*, instantiated identically on gradient descent and symbolic search.

Clause 4 (falsification): it predicts its own nulls. A regularity worth the name must predict where it *fails*. On the 79 deliberately heterogeneous worlds (E71) cross-world transfer is the random floor; on ARC the cross-task ladder is flat; only coherent families transfer—diagnosis (E74) and, on a real human-authored domain, List Functions (E84, §4.7, a +34-point exactness-gated cross-world lift)—and inside the heterogeneous set a coherence-guided search still *finds* a generalizing subfamily (companion framework paper [35]). A real-data probe of the same prediction (E83): training one adapter on 21 CLRS-Text algorithms and evaluating 8 held-out algorithms gives *no* cross-world lift—held-out accuracy does not rise above zero-shot—exactly as expected for a set of algorithms (sorting, graph search, string matching) that share little skill. We report this as a *weak* null, however: the 7B base sits near the floor on these held-out algorithms, so the test has little headroom in either direction. One null we did *not* predict in advance and report as found, not forecast: cross-language-frame transfer (E81, §4.9) gives no lift, even though the exactness clause is consistent with it (matched frames beat corrupted). The lesson is the honest one—the regularity's *form* holds where it was tested, but a held-out language (or a CLRS algorithm) is its own skill, outside the family structure the regularity scores.

The reader's guideline: how many worlds to simulate. Because the curve saturates, the optimal number of simulated worlds is its knee. From the per-world compute curve, N_{90} (worlds for 90% of the asymptotic gain) is 3 for List Functions, 9 for ARC, and 15 for CLRS—*harder domains need more simulated worlds*, but only a handful, not thousands. The honest scope: the curve's *form* appears to recur across the verifiable-world problems and learners we tested; the constants (τ, the depth cap) are learner-specific; and the predictor is cheap but not free—a small probe, localising the genuinely hard part where it belongs, in choosing a representation that makes a world's rule identifiable.

4.9 Composites with varying rules: language frames and the invariant principle (E81)

Every world so far has one rule-set. But a single computational *principle* can be expressed under many rule-sets, and world-time compute should recover the principle that is invariant across them—a reference-frame view of generalization. We make this concrete with programming: a **composite** world is one programming problem, and its **sub-worlds** are that problem realized in 5 languages (Python, C++, Java, JS, Go)—each a verified code world whose transition is that language's tests executing. A language is a *frame*: the same principle in different coordinates. We build 20 such composites from HumanEval-X and ask whether traversing the other language-frames teaches the frame-invariant principle well enough to solve a *held-out* frame. For each problem and target language we measure three arms: *zero-shot* (the base model on the held-out frame), *frame TTT* (60 steps of QLoRA on the *other* languages' prompt→solution pairs for that same problem, then evaluate the held-out frame), and *corrupt* (the same budget on *mismatched*-language pairs—the exactness control). We run two open-weight bases of differing coding strength: Qwen2.5-Coder-7B (strong) and Llama-3.1-8B (weak).

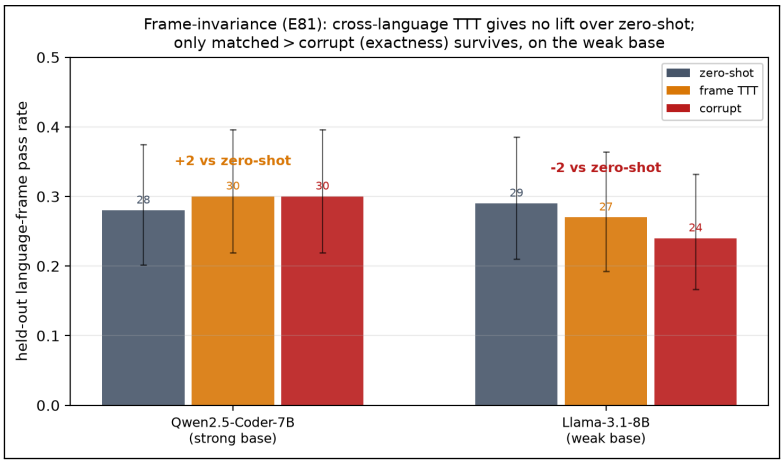

Figure 10: **Frame-invariance across programming-language frames is a negative (E81).** A composite world is one problem in 5 languages; *frame TTT* fine-tunes on the other languages and evaluates a held-out language (n=100 problem×language holdouts per base). Cross-frame TTT does not beat zero-shot on either base—the strong coder is flat (28%→30%) and the weak base drops (29%→27%); only the matched > corrupt ordering on the weak base (27% vs 24%) survives. We report this as a boundary of world-time compute. Error bars are Wilson 95% intervals.

This is the clearest *negative* in the paper, and we report it as a boundary (Figure 10). Cross-frame transfer does not beat zero-shot on either base. The strong coder moves only 28%→30% (within Wilson noise at n=100), and the *weak* base, where the headroom hypothesis predicted the largest gain, instead *drops* (29%→27%). The only structure that survives is a weak exactness ordering on the weak base—matched frames beat corrupted ones (27% vs 24%)—but even matched frames fail to recover the zero-shot baseline. So unlike the diagnosis-world (E74) and ARC (E80) settings, where world-time compute lifts held-out accuracy, traversing the language-frames of one problem does *not* teach a transferable principle: a held-out language behaves as its own skill, and a handful of TTT steps on sibling languages perturbs the model more than it informs it. The honest lesson is a limit on the reference-frame analogy— composites whose sub-worlds have genuinely different rule-sets (languages) are not automatically a single learnable frame, and exactness (matched > corrupt) is necessary but far from sufficient for transfer. The clean positive on *this* substrate comes not from cross-frame TTT but from the verified oracle used as a generator and backstop, which we isolate next (E82).

4.10 The hybrid loop: a verified world as generator and backstop (E82)

E81 shows that traversing frames for free transfer fails on code; the value of a verified world here is its *exactness*, used directly. E82 isolates that, putting all three routes (Table 10) in one open-weight, zero-human-label experiment on HumanEval-X Python, whose test suite is a perfect verifier. *Route 1 (tool as generator):* sample from the base model and keep only test-passing solutions—exact training data with no labels (76/80 kept for Qwen, 66/80 for Llama). *Route 2 (amortize):* QLoRA-tune on those verified solutions and re-measure pass@1. *Route 3 (hybrid):* the amortized model generates, the tool verifies, and on failure it retries up to 5 times. We run the same two bases as E81 over 40 held-out problems (Figure 11).

Three findings, one of them a caution. First, **amortization is not free**: self-distilling on verified solutions *helps* the strong base (80%→85%, +5 pts) but *hurts* the weak one (57%→50%, -7 pts)—the weak model's bootstrap set is sparser and noisier, and tuning on it perturbs more

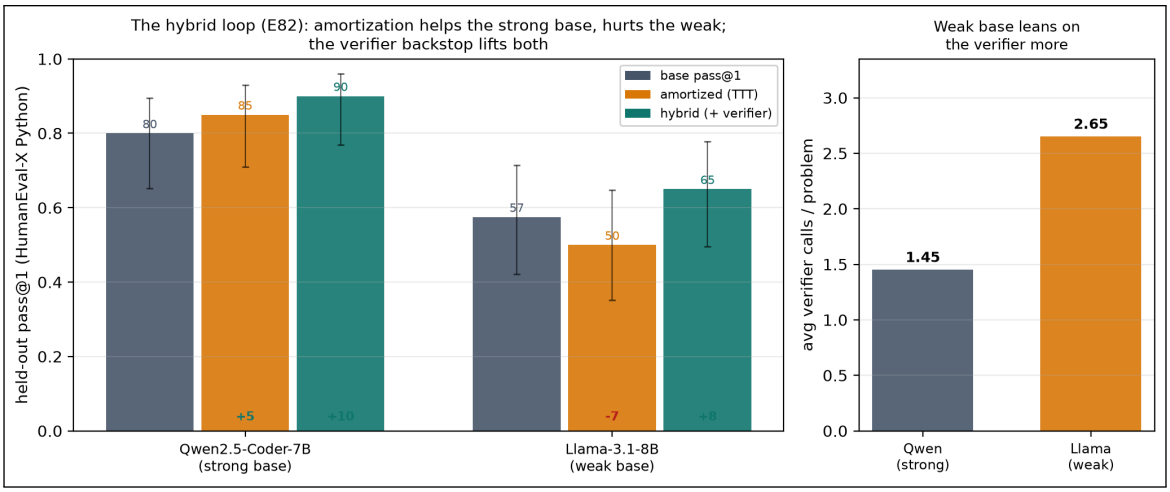

Figure 11: **The hybrid loop with open weights and zero human labels (E82).** Left: held-out pass@1 on HumanEval-X Python for base → amortized (QLoRA on self-generated test-passing solutions) → hybrid (amortized model + verify-and-retry, up to 5), with signed deltas vs. base. Amortization helps the strong base (+5) but hurts the weak one (-7); the verifier backstop lifts both (+10, +8). Right: the weak base leans on the verifier more (2.65 vs 1.45 calls per problem). Error bars are Wilson 95% intervals over 40 problems.

than it teaches. So the naive "smaller models gain more" prediction is *falsified* for pure test-time training on this task. Second, **the hybrid loop lifts both** above base (+10 pts for Qwen to 90%; +8 pts for Llama to 65%): the verifier backstop recovers what amortization alone risks, and is the robustly-positive route regardless of base strength. Third, the headroom signal that *does* survive is **verifier reliance**: the weak base calls the tool far more often inside the loop (2.65 vs 1.45 calls/problem). The practical reading maps onto the three routes: amortization (Route 2) pays off only when the base is strong enough to generate clean self-training data; the hybrid (Route 3) is the safe default; and a weaker model simply shifts more of the work onto the exact world. The exactness that E45 et al. buy, and that E81 found necessary-but-insufficient for transfer, is here *sufficient*—when the world is used as an oracle rather than a curriculum.

4.11 Against trained dynamics

The LLM next-state engine is a structural proxy; E12 adds real learned baselines trained on environment transitions (Figure 12). An MLP and a 1-NN memorizer were each trained on $K \in \{100, 1000, 10000\}$ random-policy transitions and held to the same evaluation as the synthesized program. Two results stand out. First, *sample efficiency*: even at $K = 10{,}000$ the MLP reaches only 50% exact accuracy on the branch-covering probe suite and 0% at 10× scale; the synthesized program scores 100% on both from *zero* transitions—its input is the rule text. Second, a measurement caution: at $K = 10{,}000$ the 1-NN memorizer completes 8/8 on-policy rollouts exactly while scoring only 30% on the probe suite—on-policy rollout fidelity can be *memorized* without learning the rules, which is precisely why branch-covering probes and OOD evaluation, not rollout agreement alone, are the right instruments for dynamics quality.

E19 extends both axes (Table 14). A *stronger* learned baseline—delta-state target, double capacity, lr-decayed training—does not improve on the original MLP (50% in distribution) and collapses identically out of distribution (0% at both 10× and 100×): the failure is not under-tuning but extrapolation itself. Across a 1×/10×/100× scale ladder the synthesized program is exact

everywhere (100% at 100×); the LLM next-state engine is roughly scale-*insensitive* but uniformly unreliable (∼40–50% at every scale)—it does not degrade OOD because it was never anchored to the distribution in the first place.

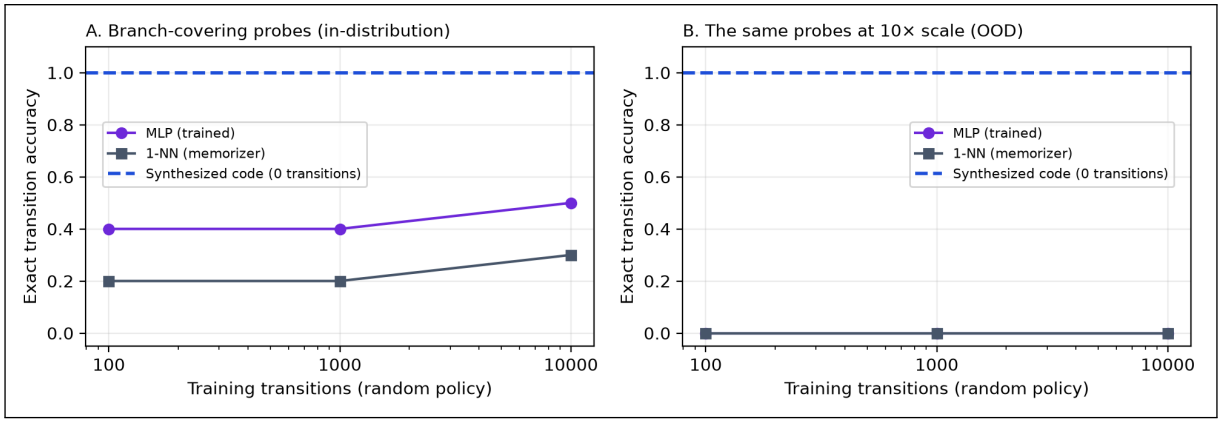

Figure 12: **Trained baselines vs. the zero-transition program (E12).** Exact transition accuracy of an MLP and a 1-NN memorizer trained on K random-policy transitions of the sprint world, on branch-covering probes in-distribution (A) and at 10× scale (B). The dashed line is the synthesized program, which uses no transitions at all. No trained baseline transfers out of distribution.

4.11.1 Induction on equal footing (E37)

A fair objection to the comparison so far is an *information asymmetry*: the synthesizer is handed the rules in natural language while the learned baselines must infer dynamics from sampled transitions. E37 removes it (Table 5). The LLM is given *only* observed $(\text{state}, \text{action}, \text{next})$ transitions—no rule text—and must induce a `transition()` program, accepted only if it reproduces the observed transitions; it is then held to the same in-dist and 10× OOD probe suites as the MLP and 1-NN trained on the identical data. Two findings result. First, on equal information code induction beats statistical learning out of distribution by a wide margin: 0.43 versus 0.00 (MLP) and 0.00 (1-NN), and the advantage is *structural*—induced-code accuracy is identical in-distribution and at 10× scale (0.43 vs 0.43, on every replicate), because the model induces symbolic rules that extrapolate, whereas the learned baselines fit magnitudes and collapse to zero OOD on every replicate. The compounding/extrapolation advantage is therefore not an artifact of being handed the rules. Second, induction is *imperfect*: the 7B model recovers only 0.43 of the probe behavior from traces (it misreads the subtle `debt`-dependent interaction), well below the rule-text anchor's 1.00. The gap between 1.00 (rules) and 0.43 (traces) is what the specification contributes—neither nothing nor everything—and is the measure of the asymmetry the headline comparison carried. Larger budgets do not close it here: the reachable transition set under a random policy saturates, so K=100 and K=1000 give the inducer essentially the same distinct examples.

Is the induction ceiling a capability limit? (E38) If ∼0.43 reflects a weak 7B generator, a stronger model should close the rules-vs-traces gap. It does not (Figure 13). Re-running E37's trace-induction across 3 generators spanning three families holds the gap roughly fixed: 0.43 for qwen2.5:7b, 0.33 for the larger qwen3-coder:30b, and 0.43 for the reasoning model gpt-oss:20b—none approaching the rule-text anchor's 1.00, and every model exactly scale-invariant (in-distribution accuracy equals 10×-OOD accuracy). Notably the 30B coder *reproduces* the observed traces best (∼0.9 of training transitions) yet generalizes no better, fitting the shown examples

Method	Information given	In-dist.	10× OOD
Code synthesis (rules)	rule text, 0 transitions	1.00	1.00
Equal information: 1000 random-policy transitions, no rule text			
Code induction	traces only	0.43	0.43
MLP	traces only	0.40	0.00
1-NN memorizer	traces only	0.17	0.00

Table 5: E37, equal-information induction: exact probe accuracy when the LLM must induce dynamics from transitions alone (no rule text), against the MLP and 1-NN on the same data, plus the rule-text synthesis anchor. Code induction is imperfect but scale-invariant (in-dist = OOD); the learned baselines collapse out of distribution; the rules-vs-traces gap measures what the specification buys.

without recovering the latent rule. The ceiling is therefore an *identifiability* limit of the data, not a capability limit: random-policy transitions do not determine the subtle interaction, so no generator can recover it from them—closing the gap needs informative transitions (active experiment design), not a bigger model. (deepseek-r1:14b is excluded: its reasoning-trace length made trace-induction impractical on the local hardware.)

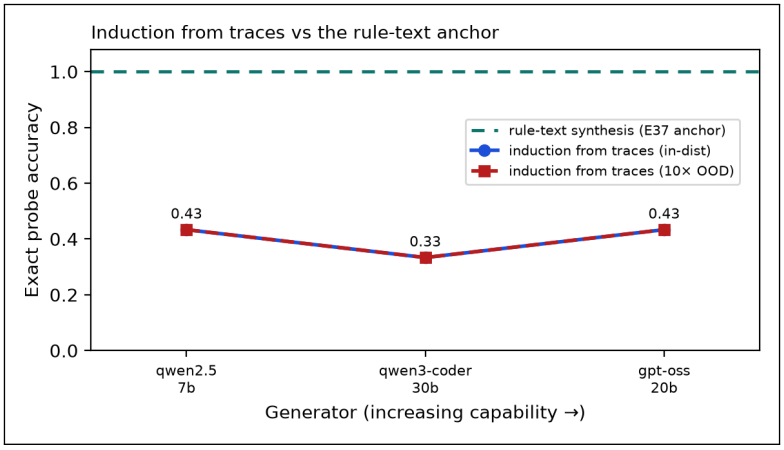

Figure 13: **Induction does not scale with generator capability (E38).** Trace-induction probe accuracy across three generator families; the rule-text anchor (E37) is 1.00. The gap is roughly flat in model size and exactly scale-invariant (in-dist = OOD)—an identifiability limit of the data, not a capability limit.

4.11.2 Acting closes the gap that scale cannot (E43)

E38 names the bottleneck—identifiability, not capability—and points to the fix: *informative* transitions rather than a bigger generator. E43 tests that fix directly. We cast dynamics recovery as version-space elimination over a candidate family of 108 sprint rules that parameterize the unknowns (ship's debt increment, the divisor k in the subtle bugs += debt ÷ k interaction, and the fix/refactor magnitudes); a candidate is eliminated the moment its predicted next state disagrees with an observed transition. We compare three probing policies on 5 hidden true rules: a *passive* random policy (the E38 setting), an *active* policy that picks the action most splitting the surviving candidates and tie-breaks toward `ship` to build the debt that makes k observable, and a *clairvoyant* reference that already knows the rule and greedily removes the most candidates per step.

Acting wins decisively (Figure 14). The active policy identifies the exact rule in 11.2 transitions on average, versus 14.7 for the passive policy—and crucially, passive observation *never* resolves 2/5

of the hidden rules within the budget, precisely the high-k rules whose interaction only surfaces at debt the random policy rarely reaches. This is the E38 ceiling reproduced as a plateau (Figure 14, left): no amount of passive data crosses it. The active policy, knowing nothing about the rule, matches the clairvoyant reference within a step on average (11.2 vs. 11.6) and occasionally beats it—the non-myopic "build debt now to disambiguate later" move is exactly what the greedy reference, optimizing per-step, fails to make. The lesson sharpens E38's: when traces underdetermine the dynamics, the leverage is in *choosing* the transitions, and a world model the agent can act inside is what makes that choice expressible.

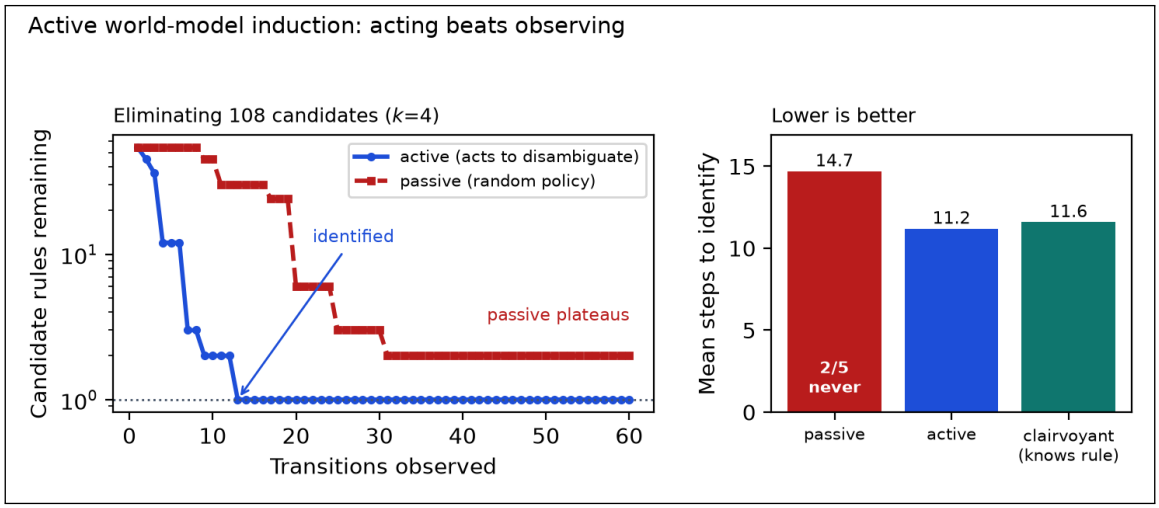

Figure 14: **Acting beats observing for rule identification (E43).** Left: candidate rules remaining (log scale) as transitions accrue, for a hidden rule the passive policy never resolves—active elimination drives 108 candidates to one while passive observation plateaus. Right: mean steps to identify across 5 hidden rules; the active policy (knowing nothing) matches the rule-knowing clairvoyant reference, while the passive policy is both slower and leaves 2/5 rules unresolved.

Further world models on the same primitives. Beyond induction, the framework expresses many additional world models that showcase expressiveness rather than carry the central argument: a non-enumerable version-space database (E46), path integrals over learning trajectories (E49), next-token world models with exact length generalization (E45), composite corporate (E48) and startup-growth (E51) worlds, a tree-of-thoughts brain simulator with real persistent memory (E58, E59), and the full perceive→world→emit→act boundary (E60). These are the subject of the companion framework paper [35].

4.12 From fidelity to decisions: planning

World models exist to improve decisions; E22 closes that chain (Table 6). A model-predictive planner runs exhaustive depth-3 lookahead inside its world model and executes the best action in the ground-truth environment. Planning through the verified synthesized program scores 7.5—*identical* to planning through the ground truth itself (7.5), as bit-exactness guarantees—against 5.0 for a reactive heuristic and 3.8 for random actions, with the whole 12-step episode planned in 0.03 s. Planning through the LLM next-state engine is confined to depth 2 by cost (86 s per episode) and scores 0.0—*worse than no model at all*: lookahead through hallucinated transitions steered the agent away from productive actions entirely. We note one honest caveat: the LLM engine falls back to a no-op when a reply does not parse as a next state, so this score conflates genuinely hallucinated

transitions with unparseable outputs; the script now logs the parse-failure rate so the two can be separated on re-run, but the qualitative conclusion—planning through an unverified model is worse than not planning—does not depend on the split. Fidelity and throughput are not abstract virtues; they are the difference between planning that helps and planning that harms.

Policy	Episode score	Planning time (s)
Lookahead d=3 via synthesized code	7.5	0.03
Lookahead d=3 via ground truth (bound)	7.5	0.00
Lookahead d=2 via LLM next-state	0.0	85.85
Reactive heuristic (no model)	5.0	0.00
Random policy (5 seeds, mean)	3.8	—

Table 6: E22: model-predictive lookahead on the sprint world, plans executed in the ground-truth environment. Value function: shipped − bugs − 0.5·debt after 12 steps.

4.13 Crossing species: verified vs. trained vs. perceptual world models (E63, E65, E67)

The comparisons so far pit verified code against an LLM next-state proxy; the fairer question a model-based-RL reader asks is how it fares against world models *trained* on environment transitions, judged on the only thing an agent cares about—task return. We run the head-to-head across 6 world models we can execute locally, on 2 symbolic domains (sprint, triage) under one battery (Table 7): every entry is a next-state predictor over the *same* state, task, and budget (K=10,000 transitions, 5 seeds), spanning the standard families—memorization (1-NN), a tabular model with nearest-neighbour backoff, linear least-squares, a Koopman/EDMD operator, and a neural MLP. The result is uniform: the verified code world model is exact in- and out-of-distribution, rolls out bit-exactly from *zero* transitions, plans optimally (return 7.5) at 2,250,984 steps/s, while *every* learned model collapses to 0.00 exact accuracy at 10× OOD and plans at or below model-free control—some *negative*, i.e. planning through a trained model is *worse than not planning*, because lookahead drives the agent into states it never saw (E61 isolates this against a single trained MLP over 5 seeds; the LLM proxy completes 0 rollouts and scores 0).

Why not Dreamer, V-JEPA, Genie, Sora? Those are a different *species*: *perceptual* world models that learn latent or pixel/video dynamics (DreamerV3 [16], MuZero [34], IRIS [24], DIAMOND [3], Genie [5], Sora [31], Fei-Fei Li's World Labs [12], and the joint-embedding V-JEPA [22]). They predict observations or embeddings, not a symbolic next state, and cannot run on these tasks without a learned perception stack, so a head-to-head would not be apples-to-apples; we compare them on *properties* in Table 1 (9 such systems) rather than fabricate a shared number. This is the genus/species distinction drawn in the introduction: the bake-off is the within-species comparison; the related-work table is the cross-species one. The honest scope of E63 is the symbolic regime—orchard is excluded because its per-agent nested-dict state is not a flat vector for the learned baselines.

A shared benchmark where the species can meet: MiniGrid (E65). The bake-off above is deliberately *within*-species (perceptual models cannot run on flat symbolic state); E65 closes that gap by moving to a benchmark where the species *can* meet: MiniGrid DoorKey-6x6. We express the task as a verified OpenWorld transition and validate it *bit-for-bit* against the real Farama `minigrid` environment (600/600 steps; `bench/validate_minigrid.py`), so the learned and perceptual models

World model	Train data (transitions)	Probe in-dist	Probe 10×OOD	Rollout exact	Control return	Audit.
verified code (CWM)	0 (rules)	1.00	1.00	1.00	+7.5	✓
1-NN	10,000	1.00	0.00	0.77	+5.7	–
tabular	10,000	1.00	0.00	0.77	+5.7	–
linear	10,000	0.46	0.00	0.19	-7.4	–
koopman	10,000	0.75	0.00	0.44	+5.6	–
MLP	10,000	0.70	0.00	0.19	-2.6	–
LLM next-state†	rules	–	–	0.00	+0.0	–

Table 7: **World-model bake-off (E63).** Every world model we can run locally, on shared symbolic tasks; probe/rollout averaged over sprint and triage (5 seeds, K=10,000). †LLM rows from E10/E11/E22. Verified code is the only model exact, OOD-robust, optimal-for-control, auditable, *and* zero-data. Perceptual world models (Dreamer/V-JEPA/Genie/Sora/...) are a different species, compared on properties in Table 1.

consume the *same* environment's rendered 48×48 pixels while OpenWorld plans over the verified code (Table 8). OpenWorld solves the task *zero-shot* with an optimal 11-step plan and no training data. DreamerV3 [16], learning a latent world model from pixels on a dedicated A100, reaches the same 100% solve rate—but only after a first success at 9,640 environment steps and convergence over ∼50k. V-JEPA-2 [22] enters as the perceptual species: with no symbolic policy it has no solve rate, so we report its representation drift (mean consecutive-frame cosine 0.48) and mark it, honestly, as not directly comparable. The reading holds the paper's thesis on the learned models' *own* terrain (pixels): verification buys the exact solution for zero data and a guarantee, where the learned model needs ∼10k pixel-level interactions merely to first succeed. DoorKey is the *image-based, few-step* end of the boundary E80 maps (§4.6): on a visual task with a short solution horizon the verified model dominates both learned species, whereas on a visual task that hinges on *rich perceptual concept abstraction* (Bongard-RWR) our symbolic world-time-compute recipe instead stalls at chance. The dividing line is consistent—verified world models win where the *reasoning*, not the perception, is the hard part and the horizon is short.

World model	Species	Train data	Solve	Cost to solution
OpenWorld	verified code	0	100%	0-shot plan, length 11
DreamerV3	learned (pixels)	9,640 steps	100%	first solve @ 9,640 steps
V-JEPA-2	perceptual	pretrained	–	cos-drift 0.48 (not a solve rate)

Table 8: **Trained vs. verified on a shared benchmark (E65).** MiniGrid DoorKey-6x6, with OpenWorld's transition validated bit-for-bit against Farama `minigrid`; DreamerV3 (1 seed, A100) and V-JEPA-2 consume the same environment's rendered pixels. V-JEPA is perceptual (no symbolic policy), hence a representation-drift metric rather than a solve rate.

Continuous control on the learned models' home turf: cartpole (E67). MiniGrid (E65) is a gridworld, which arguably favors symbolic models. E67 moves the same three-species contest to *continuous control*—cartpole-swingup from 64×64 pixels—where learned and perceptual world models are strongest, yet OpenWorld can still write the dynamics (the standard cartpole equations of motion) as verified code, validated bit-identical against an independent reference. The picture holds, and sharpens (Table 9). OpenWorld plans the swing-up with CEM-MPC and solves 100% of episodes with *zero* data (random control 0%). DreamerV3, learning a latent world model from pixels end to end, reaches reward 120±9 and first swings up around 8.8k±2.4k environment steps

(mean±sd over 3 seeds)—about 5× more sample-efficient than DreamerV2. The perceptual species (V-JEPA-2 and other frozen encoders) scores 0% through *every* controller we tried. That last result is the interesting one, and it is not what it first looks like: the frozen features encode the full state perfectly well; the failure is *control*, not perception. We trace it in the companion framework paper [35].

World model	Species	Train data	Result
OpenWorld	verified code	0	100% solve (CEM-MPC, 0-shot)
DreamerV3	learned (pixels)	8.8k steps	reward 120±9; first solve 8.8±2.4k steps (3 seeds)
DreamerV2	learned (pixels)	56k steps	reward 99 (∼5× less sample-efficient)
V-JEPA-2 & frozen enc.	perceptual (frozen)	pretrained	0% solve (every controller)
random control	–	–	0% solve

Table 9: **Cartpole-swingup head-to-head (E67).** Verified-code vs. end-to-end learned vs. frozen-perceptual world models on continuous control from pixels. OpenWorld's ODE is validated bit-identical to an independent reference; DreamerV3/V2 trained on a dedicated A100. Learned models report converged reward + sample efficiency; OpenWorld and the frozen-perceptual models report a swing-up solve rate.

4.14 Scope boundaries and oracle-free verification

Three round-3 experiments chart where the paradigm holds and what to do at the edge.

The complexity cliff (E20). Parametric worlds with R interacting rules (conditions reading pre-action state, summed effects, clamping) were synthesized from rule text at $R \in \{4, 8, 12, 16\}$ (Figure 15). Mean probe accuracy falls from 1.00 at R=4 to 0.53 at R=8, 0.25 at R=12, and 0.15 at R=16: monolithic single-function synthesis with a 7B generator degrades sharply once roughly eight coupled rules must be implemented at once. This is the paradigm's present boundary at this model scale, and it motivates compositional synthesis—one expert per rule, as in PoE-World [32]—as the structural fix.

Stochastic dynamics (E21). Threading a seed through the state (`rng_seed` in, derived seed out) extends verified synthesis to stochastic worlds without sacrificing replayability. On a queueing world with a declared 0.3 arrival probability, accepted programs (67% of attempts) matched the declared distribution within 1.2 percentage points over 2,000 seeded transitions, kept the deterministic bookkeeping perfect (100%), replayed bit-exactly, and in fact matched the hand-written stochastic oracle bit-for-bit (100%).

Oracle-free error detection (E23). Practitioners lack the hand-written oracles our evaluations use; they can, however, synthesize the dynamics several times. Across 30 stored programs and 900 program–probe pairs, disagreement with the ensemble majority detected ground-truth errors with 100% precision and 100% recall—the majority vote reconstructed the oracle exactly on these worlds—and a program's agreement rate ranked its true accuracy perfectly (Spearman $\rho = 1.00$). The caveat is structural: majority-as-oracle fails under correlated errors (e.g., every model misreading the same ambiguous rule), so ensemble disagreement is a cheap screen, not a proof; it composes naturally with the invariant and critic gates.

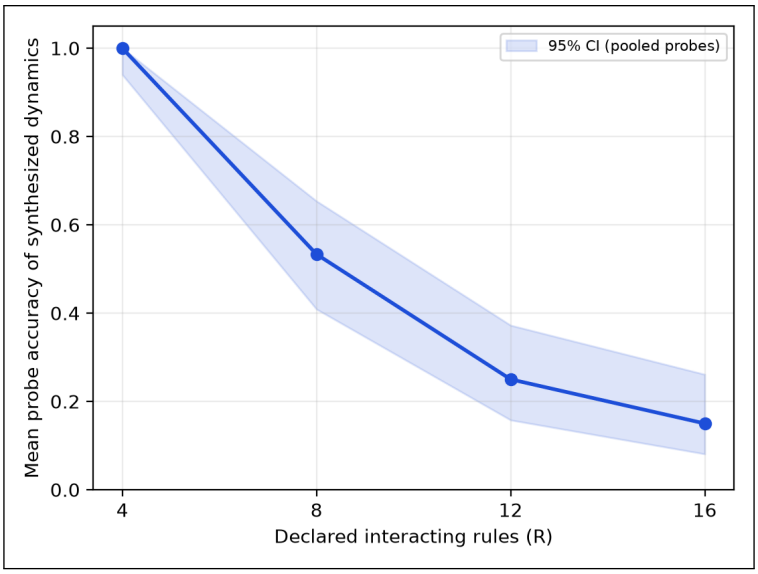

Figure 15: **The complexity cliff (E20).** Probe accuracy of 7B-synthesized dynamics versus the number of declared interacting rules; three syntheses per point, shaded band is the 95% CI over pooled probes. Monolithic synthesis is reliable at R=4 and unreliable past R≈8.

5 Discussion and Limitations

Interpretation. The experiments quantify a structural trade. Learned world models buy pixel-native breadth with training compute and pay in compounding error, opacity, and frozen values. Verified code synthesis buys exactness, speed, OOD transfer, and auditability, and pays at the boundary of what can be declared symbolically. Within that boundary the results are one-sided: a laptop-scale local model, orchestrated as plan–generate–verify, produces simulators that are *perfect* where learned-style baselines are approximate. The practical workflow the ablations license: declare rules precisely; verify with invariants; probe the accepted artifact against intent; tighten and recompile. Each gate buys measurable correctness, and the artifact remains a diffable program at every stage.

Is world-time compute just fine-tuning? Mechanically, yes: there is no new optimizer or architecture, only LoRA-SFT. The assertion is about the *data source*, not the optimizer. Ordinary fine-tuning needs a labelled corpus—scarce, costly, and sometimes wrong; world-time compute instead fine-tunes on trajectories from a *verified* world model, which generates unlimited examples and labels them *exactly* for free, because the transition is code and the oracle is execution. In one line: a correct world model turns compute into unlimited correct training data. Whether the "verified" qualifier is load-bearing—rather than decoration on plain SFT—is an empirical question with a clean discriminator, the **corrupted-label ablation**: randomising the labels keeps the fine-tuning and the world's structure but destroys their correctness, so if accuracy is unchanged the framing adds nothing. It does collapse to the no-fine-tune floor: the corrupted-demonstration arm on real ARC worlds falls to 2% against the verified arm (§4.5, Fig. 7). So the contribution is conceptual, not algorithmic, and only two things earn the approach its name over "SFT with extra steps": (i) this verified-label dependence, and (ii) *cross-world* transfer—training on a *family* of worlds lifts accuracy on *held-out* worlds never fine-tuned on (E74, E76), which same-world fine-tuning cannot do. Where neither holds—heterogeneous worlds with no shared skill (E71, E73), or

Route	Exact?	Training	Tool at inference	Generalises to new worlds
1. Tool	yes	none	yes	n/a (use any world)
2. Per-world TTT	approx.	QLoRA, one world	no	within the world
3. World-time compute	approx.	QLoRA, many worlds	no	yes (across the family)
Hybrid	on fallback	QLoRA	only when unsure	yes, with exact backstop

Table 10: **Three routes to use a verified world model**, plus the hybrid. Need exactness/audit → Route 1; amortise one world → Route 2; generalise a family → Route 3; best of both → Hybrid.

a benchmark of per-task-novel rules where cross-task transfer stays weak (ARC, §4.5)—it is, fairly, just expensive fine-tuning, and we say so. This places world-time compute squarely in the *self-training* lineage—STaR [45], ReST and ReST-EM [13, 36], expert iteration [4], rejection-sampling fine-tuning [44], and learning from verifiable rewards [17]—all of which retrain a model on its own outputs filtered by a correctness check. Two differences are the contribution, not the loop: the verifier is a *full world model with dynamics*, so the training unit is a verified *trajectory* (a state-action-next-state transition), not a checked final answer; and the target is a *different generalization axis*—held-out *worlds* in a family, rather than the same task improved in place. In those terms the corrupted-label ablation is precisely a verifier-reliability study: the failure mode (training on confidently-wrong self-labels) that self-training most fears, here ruled out by construction.

When does it pay off? Few-step reasoning. Pulling the results together (§4.6) yields a single operating regime. Verified world models and world-time compute win where a task reduces to *few reasoning steps* over a state that can be *told* symbolically: shallow rule induction (List Functions, the largest lift), small grid transforms (ARC), and short-horizon planning (MiniGrid DoorKey, solved zero-shot while DreamerV3 needs 10^4–10^5 interactions). Performance degrades along two axes—as the required reasoning *lengthens* (CLRS algorithm traces, the lowest of the symbolic domains; and the E20 complexity cliff past ∼8 composed rules), and as the latent rule must be *induced from rich perception* rather than read from symbolic state (Bongard-RWR vision, flat at chance). This is not a hedge but the contribution's shape: a verified, auditable, zero-data lever that is strongest precisely where today's data-hungry learned world models are weakest—short, specifiable reasoning—and that cleanly cedes the long-horizon, perception-heavy regime to them.

Three routes to use a world model. A verified world model is a reusable artifact, and it is useful whether or not one ever fine-tunes; world-time compute is one of three ways to spend it (Table 10). *Route 1—as a tool:* serve the world and call it—plan through it, query exact next-states, or use it as a verifier (no training; exact, auditable; §4.12, §4.13). *Route 2—per-world test-time training:* distil a single world's exactly-labelled trajectories into weights with QLoRA, amortising that world's skill for tool-free inference (§4.5). *Route 3—world-time compute:* traverse a *family* of verified worlds and fine-tune to generalise to held-out worlds (§4.3, §4.8). The *hybrid* composes them: use the tool to *generate* exact trajectories, training to *internalise* them, and the exact tool as a fallback for high-stakes queries—the tool bootstraps the data, training amortises it, and exactness is recoverable on demand.

Value alignment as an open specification. The dial experiments ground the alignment claim: because objectives are declared artifacts, the welfare–fairness frontier is *operational*—an operator moves along it in real time, and a tuner searches moral configurations jointly with world design,

surfacing the manifold of value settings that solve a task rather than a single frozen compromise. Two qualifications, owed to the philosophical review. First, openness is only as wide as the expressible structures: a weighted sum is act consequentialism, and the companion framework paper [35] shows that aggregator choice, side constraints, and theory uncertainty are distinct moral objects the framework must (and now does) represent rather than approximate with weights. Second, open specification *manages* the specification trap [38] procedurally rather than resolving it: the trap relocates from frozen weights to the operator's hand on the dial, and which hand that should be—justifiability to those affected, in the contractualist's sense [33]—is a question no architecture settles. The tuner's solving manifold is at best a crude contractualist surface: the set of configurations meeting every stakeholder's declared minimum.

Limitations. (i) Scope: symbolic-state worlds; pixel-native domains remain the territory of learned models—the comparison here is within the symbolic regime, and neither the LLM proxy nor the numpy baselines is a trained Dreamer. The headline comparison also gives the synthesizer the rule text; E37 shows the structural OOD advantage survives removing that asymmetry (induction from traces alone), but induction is then imperfect (0.43 vs the rule-text 1.00), so part of the headline margin reflects the value of an explicit specification, not the representation alone. (ii) Complexity: monolithic synthesis with a 7B generator is reliable at four interacting rules and unreliable past roughly eight (E20); the headline worlds sit below that cliff, and richer worlds will need compositional synthesis [32] or larger generators. (iii) Scale: twenty repair tasks and three handwritten worlds; paired tests sharpen the comparisons, but the benchmark remains an archetype suite, not HumanEval [6], 3B+ agents saturate it, and the classic defect archetypes are plausibly present in pretraining data: agent solve rates should not be read as measurements of debugging skill—the world-model claims are unaffected, since the environment executes exactly regardless of what the agent has memorized. (iv) Synthesis replicates across three model families (E16), but the judge, critic, and repair-agent roles remain Qwen-only. The synthesis, judge, and repair experiments run on one consumer machine under the stated quantizations; the fine-tuning experiments (E74, E78, E80–E83) instead run on a single cloud A100 (4-bit QLoRA for the 7B and larger models), which we state plainly so the consumer-hardware framing is not read as covering the training runs. (v) Judges: selection accuracy and order-consistency are measured but imperfect, the pooled selection margin is marginal ($p = 0.078$) and costs $\sim 4\times$ inference, and rubric grading showed compression; judge verdicts should gate, not replace, programmatic verification. (vi) Ensemble self-checking (E23) achieved perfect precision/recall here but is structurally vulnerable to correlated errors across generators reading the same ambiguous rule; it is a screen, not a proof. (vii) The sandbox is best-effort isolation against accident, not an adversarial security boundary (generated patches twice defeated in-process timeouts before the fork-based runner; see the repository history). (viii) The moral machinery, even broadened, evaluates declared values; whose values are declared—and the legitimacy of the declaring hand—is a procedural question outside any architecture [33], and the parliament's credences are themselves operator-set dials. (ix) Several experiments were redesigned after pilots and four internal review rounds (E3, E5/E6, E11–E27); pilot saturation results are reported alongside the final protocols, and all reviews ship in the repository. (x) Multiple comparisons: across 78 experiments we report many p-values and rank correlations and apply *no* family-wise or false-discovery correction; with this many tests the borderline results (p between 0.05 and 0.10—notably the judge margin and the rubric-paraphrase correlation) should be read as exploratory, and only the large deterministic effects (exact-match rates, OOD collapse, the complexity cliff) are robust to the multiplicity. (xi) Variance: the deterministic claims are exact-match against oracles and need no error bars, but several *stochastic* (LLM) headline quantities are point

estimates from small, single-seed samples (e.g. the E1 divergence step, n=3, one world) without across-seed variance; we add the pooled multi-world view (E11) where it matters, but per-model probe accuracies should be read as estimates, not tight measurements. This caveat extends to the headline world-time-compute results: the E74 model-size scaling is a single fine-tuning seed per size, and the real-data ARC/CLRS arms are single-seed at $n\approx$30–40 with CIs that touch the floor, so the size-trend and the weaker real-data domains should be read as exploratory; for ARC we additionally report a three-seed re-run of the verified-vs-corrupt gap (across-seed CIs). (xii) Two illustrative studies are true *by construction*: E59's architecture lift follows arithmetically from a modeled fixed-capability backbone, and E60's routing gap over exact-key lookup is guaranteed by the comparison design (we add a fair lexical baseline and report the modest gap over it); both are flagged in situ as engineering demonstrations, not measured effects. (xiii) Porting to real data surfaces two validity conditions: the input representation must encode *per-world* structure (a ProteinGym port describing each mutation only by global biochemical deltas made every assay-world identical and flattened the world-count curve until per-world local-sequence context was restored), and faithful data loading—e.g. the wild-type sequence and one-based mutation indexing—is the dominant correctness risk. We treat both as the first checks any real-domain port must pass. (xiv) The distinction we draw from self-training—that the training unit is a verified *trajectory* (s, a, s') rather than a checked final answer—is argued conceptually but not yet isolated experimentally; a head-to-head of fine-tuning on trajectories vs. on verified prompt→answer pairs from the *same* worlds is the cleanest test, and is future work.

6 Conclusion

A training-free symbolic world model—synthesized as code by a local model and verified before acceptance—is bit-exact where learned-style dynamics compound error, and plans *as well as the ground truth itself* where planning through a hallucinating model is worse than no model. And beyond serving as a simulator to plan *in*, the same verified worlds are training *signal*: traversing many world models of a domain lifts a model's generalization to held-out worlds, scaling with the number of worlds traversed—*world-time compute*, a training-time analogue of test-time compute. Its boundary is equally measured: monolithic 7B synthesis fails past roughly eight interacting rules. The world-model field's trajectory has been to scale parameters until hallucination is suppressed; these results argue for an orthogonal path—make the model a program, verify the program, and keep the values dialable. Next steps: compositional synthesis to cross the complexity cliff [32], the coding world at standard-benchmark scale, hybrid perception-to-symbol pipelines, and judges that propose—not just select—world repairs.

Declaration of generative AI use

During the preparation of this work the author used *Claude Fable 5 (Anthropic, via Claude Code)* to implement the OpenWorld framework and experiment scripts, execute the experimental campaign, generate figures and tables, and draft this manuscript, under the author's direction. The author reviewed and edited the content and takes full responsibility for the content of the publication. All experimental results are produced by the released deterministic scripts (not by generative-AI estimation), and no AI tool is listed as an author, as such tools do not satisfy authorship criteria.

Data and code availability

All code, the benchmark suite, experiment scripts, raw result JSON, and this manuscript's build are available at github.com/quome-cloud/openworld. All data are synthetic; no human-subject or patient data were used.

Author contributions

J.S. conceived the study, directed the implementation and experiments, and reviewed the manuscript.

Competing interests

The author declares no competing interests.

Funding

This work received no specific external funding.

A Index of experiments

The experiments E1–E82 are introduced across the Results section in thematic rather than numeric order; this index lists each by number with a one-line description and the section, figure, or table where it appears, as a navigation aid. Numbers are non-contiguous: E14 and E53 are retired (absorbed or cut) and do not appear in the text; E28–E29 are described in the Experimental Setup but their dedicated results section was cut; and E66, E69–E73, E75, E78, and E79 were not used (a superseded probe or a negative result excluded from the paper). The world-time-compute family E74–E82 (E80 spans several real-data domains and the descriptive regularity) carries the central result.

Table 11: Index of experiments E1–E82. E14, E53 retired and E66/E69–E73/E75/E78/E79 unused (absent from the text); E28–E29 are described in the Experimental Setup but their results section was cut.

E#	Description	Where
E1	Head-to-head symbolic vs. learned dynamics on the sprint world	Tab. 2
E2	Synthesis reliability: five compile attempts per world	Tab. 12
E4	Head-to-head engine comparison, primary sprint result	Tab. 2
E9	Configuration-search strategies on triage auto-tuning	Tab. 15
E10	Head-to-head primary comparison, throughput and accuracy	Tab. 2
E11	Multi-world fidelity replication, eight scripts per world	Tab. 3
E12	Trained MLP/1-NN baselines vs. the zero-transition program	Fig. 12
E14	*Retired (absorbed); not in text*	—
E16	Cross-family synthesis replication (Llama, Gemma)	Tab. 12
E18	Filter-vs-repair decomposition of the verification gate	Tab. 13
E19	Scale-ladder vs. a stronger learned baseline, OOD collapse	Tab. 14
E20	The complexity cliff: synthesis degrades past eight rules	Fig. 15
E21	Stochastic dynamics via seed-threaded verified synthesis	§4.14
E22	Planning: model-predictive lookahead through the program	Tab. 6
E23	Oracle-free error detection by ensemble disagreement	§4.14
E28	*Cut (setup only)*: atomic SWE-style repair suite	§ Setup

continued on next page

E#	Description	Where
E29	*Cut (setup only)*: staged latent-defect repair suite	§ Setup
E37	Equal-information induction: code beats learning OOD	Tab. 5
E38	The induction ceiling is identifiability, not capability	Fig. 13
E43	Active induction: acting closes the identifiability gap	Fig. 14
E53	*Retired (sheaf section cut); not in text*	—
E61	Verified code vs a trained world model on control (absorbed into E63)	Tab. 7
E63	World-model bake-off: 6 runnable models × 2 domains	Tab. 7
E65	Cross-species on MiniGrid: verified code vs. DreamerV3 / V-JEPA-2	§4.13
E67	Continuous control: cartpole-swingup vs. learned/perceptual models	§4.13
E74	World-time compute: held-out lift vs. model size (diagnosis family)	§4.3
E76	World-count scaling of the world-time-compute lever	§4.4
E77	Coding-world transfer of world-time compute (pass@5)	§4.4
E78b	Label-exactness ablation isolates the causal driver	§4.4
E80	Real-data world-time compute on ARC-AGI (per-world TTT + ladder)	§4.5
E80	Replication across List Functions, CLRS, Bongard-RWR (vision boundary)	§4.6
E80	A descriptive regularity: LODO fit, tabular out-of-sample, learner-invariance	§4.8
E81	Frame-invariance across language frames (clean negative)	§4.9
E82	The hybrid loop: verified world as generator + inference backstop	§4.10
E83	Real-data cross-world transfer on CLRS-Text (weak null)	§4.8

B Detailed result tables

Generator (family)	Runs	Verified acceptance (95% CI)	Probe acc. of accepted	Mean synthesis time
qwen2.5:7b (Qwen)	15	100% [0.80, 1.00]	0.98	—
qwen2.5:3b (Qwen)	15	100% [0.80, 1.00]	0.87	—
llama3.1:8b (Llama)	9	100% [0.70, 1.00]	0.85	9s
gemma2:9b (Gemma)	9	100% [0.70, 1.00]	1.00	4s

Table 12: E2 + E16: synthesis reliability by generator scale and family. Qwen rows: five compilation attempts per world; Llama/Gemma rows: three per world. Maximum four verify–repair iterations each; wall-clock includes all iterations.

Regime	Acceptance	Probe acc. (accepted)	Probe acc. (all runs)
Filter only (max_iters=1)	62%	0.62	0.39
Filter + repair loop (max_iters=4)	100%	0.65	0.65

Table 13: E18: filter-vs-repair decomposition of the full verification gate (3B generator, eight paired high-temperature attempts per regime).

Engine	1×	10×	100×
Synthesized code (0 transitions)	10/10	10/10	10/10
Delta-MLP, 128h (10k transitions)	5/10	0/10	0/10
MLP, 64h (10k transitions)	5/10	0/10	0/10
1-NN memorizer (10k transitions)	3/10	0/10	0/10
LLM next-state (7B, 0 transitions)	4/10	4/10	5/10

Table 14: E19: exact transition accuracy on the branch-covering probe suite across a 1×/10×/100× scale ladder.

Strategy	Seeds solving	Mean trials to first solve	Mean best score
random	10/10	4	17.60
tpe	10/10	5	17.60
random+refine	10/10	4	17.60

Table 15: E9: tuning-strategy comparison on the triage auto-configuration problem; ten seeds per strategy, 200-trial budget.

C The synthesized dynamics artifact

A representative accepted sprint-world transition, synthesized by `qwen2.5:7b` and verified by the full gate—the artifact whose exactness underwrites Table 2:

Accepted `transition()` (verbatim)

```
def transition(state, action):
    next_state = dict(state)
    name = action["name"]
    if name == "ship" and next_state["backlog"] > 0:
        next_state["backlog"] -= 1
        next_state["shipped"] += 1
        next_state["debt"] += 1
        next_state["bugs"] += next_state["debt"] // 4
    elif name == "fix":
        next_state["bugs"] = max(0, next_state["bugs"] - 2)
    elif name == "refactor":
        next_state["debt"] = max(0, next_state["debt"] - 2)
    return next_state
```

D Example tasks and failure cases

The coding-world tasks pair a buggy function with hidden tests; e.g. `dedupe` returns `list(set(xs))` (order-destroying) and must preserve first-seen order, and `balanced` counts parenthesis depth but misses early-imbalance strings like `")("`. Failure modes observed in the campaign: the 1.5B baseline resubmitted near-identical patches until the attempt budget expired on tasks where its first reading of the spec was wrong—precisely the local minimum that high-temperature candidate sampling plus judge selection escapes; conversely, when several candidates passed, the judge's pick was strongly order-sensitive (E13: raw choices matched under order reversal on only 28% of rounds) while its accuracy was not—evidence that judge criteria and presentation order need auditing even when outcomes look fine. In E7/E15, the rubric judge compressed mid-range episodes toward similar scores (rank correlation 0.75 rather than 1.0, dropping to 0.58 under paraphrase), consistent with

known LLM-judge coarseness [46].

E Reproducibility

One pipeline rebuilds every artifact: experiment scripts in `experiments/` write `results/*.json` (seeds fixed in-source); `python3 scripts/make_paper_assets.py` regenerates every figure, table, and the `numbers.tex` macros from those JSON files; and `make` in `paper/` builds this PDF. A continuous-integration workflow runs the offline test suite, re-runs the asset pipeline, and fails if `numbers.tex` would change, so the paper's numbers cannot silently drift from the committed results. `experiments/MANIFEST.md` classifies every experiment as deterministic-offline or Ollama-dependent and records its model and provenance; `experiments/requirements.lock` pins the analysis stack (Python, numpy, matplotlib). The 8 Ollama model snapshots that appear across the runs are `gemma2:9b`, `gpt-oss:20b`, `llama3.1:8b`, `qwen2.5:1.5b`, `qwen2.5:3b`, `qwen2.5:7b`, `qwen2.5:14b`, `qwen3-coder:30b` (quantization `Q4_K_M`); platform, versions, and timestamp are recorded in each result file. Deterministic-offline experiments reproduce bit-for-bit; the Ollama-dependent runs reproduce *in distribution* (Metal is not bit-deterministic even at fixed seed), with dataset/model-pinned frozen artifacts where exact numbers are cited.

References

[1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019.

[2] Ekin Akyürek, Mehul Damani, Linlu Qiu, Han Guo, Yoon Kim, and Jacob Andreas. The surprising effectiveness of test-time training for abstract reasoning. *arXiv preprint arXiv:2411.07279*, 2024.

[3] Eloi Alonso, Adam Jelley, Vincent Micheli, et al. Diffusion for world modeling: Visual details matter in Atari. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[4] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[5] Jake Bruce, Michael Dennis, Ashley Edwards, et al. Genie: Generative interactive environments. *arXiv preprint arXiv:2402.15391*, 2024.

[6] Mark Chen, Jerry Tworek, Heewoo Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[7] François Chollet. On the measure of intelligence. *arXiv preprint arXiv:1911.01547*, 2019.

[8] François Chollet, Mike Knoop, Gregory Kamradt, and Bryan Landers. ARC prize 2024: Technical report. arXiv preprint arXiv:2412.04604, 2024.

[9] François Chollet, Mike Knoop, Gregory Kamradt, Bryan Landers, and Henry Pinkard. ARC-AGI-2: A new challenge for frontier AI reasoning systems. arXiv preprint arXiv:2505.11831, 2025.

[10] Karl Cobbe, Christopher Hesse, Jacob Hilton, and John Schulman. Leveraging procedural generation to benchmark reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.

[11] Nicola Dainese, Matteo Merler, Minttu Alakuijala, and Pekka Marttinen. Generating code world models with large language models guided by Monte Carlo tree search. *arXiv preprint arXiv:2405.15383*, 2024.

[12] Fei-Fei Li and World Labs. Large world models and spatial intelligence. World Labs, 2025.

[13] Caglar Gulcehre, Tom Le Paine, Srivatsan Srinivasan, Ksenia Konyushkova, Lotte Weerts, Abhishek Sharma, Aditya Siddhant, Alex Ahern, Miaosen Wang, Chenjie Gu, et al. Reinforced self-training (ReST) for language modeling. *arXiv preprint arXiv:2308.08998*, 2023.

[14] Daya Guo, Dejian Yang, Haowei Zhang, et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[15] David Ha and Jürgen Schmidhuber. World models. *arXiv preprint arXiv:1803.10122*, 2018.

[16] Danijar Hafner, Jurgis Pasukonis, Jimmy Ba, and Timothy Lillicrap. Mastering diverse domains through world models. *arXiv preprint arXiv:2301.04104*, 2023.

[17] Nathan Lambert, Jacob Morrison, Valentina Pyatkin, Shengyi Huang, Hamish Ivison, Faeze Brahman, Lester James V. Miranda, Alisa Liu, Nouha Dziri, Shane Lyu, et al. Tülu 3: Pushing frontiers in open language model post-training. *arXiv preprint arXiv:2411.15124*, 2024.

[18] Wolfgang Lehrach, Joseph Marino, Luis Piloto, et al. Code world models for general game playing. *arXiv preprint arXiv:2510.04542*, 2025.

[19] Wen-Ding Li, Keya Hu, Carter Larsen, Yuqing Wu, Simon Alford, Caleb Woo, Spencer M. Dunn, Hao Tang, Wei-Long Zheng, Yewen Pu, and Kevin Ellis. Combining induction and transduction for abstract reasoning. *arXiv preprint arXiv:2411.02272*, 2024.

[20] Larisa Markeeva, Sean McLeish, Borja Ibarz, Wilfried Bounsi, Olga Kozlova, Alex Vitvitskyi, Charles Blundell, Tom Goldstein, Avi Schwarzschild, and Petar Veličković. The CLRS-Text algorithmic reasoning language benchmark. *arXiv preprint arXiv:2406.04229*, 2024.

[21] MAS Authors. Moral anchor system: A predictive framework for AI value alignment and drift prevention. *arXiv preprint arXiv:2510.04073*, 2025.

[22] Meta AI (FAIR). V-JEPA 2: Self-supervised video world models from joint-embedding predictive architectures. arXiv preprint, 2025. Joint-embedding predictive world model; predicts in representation space, not pixels.

[23] Meta FAIR CodeGen Team. CWM: An open-weights LLM for research on code generation with world models. *arXiv preprint arXiv:2510.02387*, 2025. 32B open-weights neural code world model; mid-trained on Python execution and Docker container traces, then multi-task RL.

[24] Vincent Micheli, Eloi Alonso, and François Fleuret. Transformers are sample-efficient world models. In *International Conference on Learning Representations (ICLR)*, 2023.

[25] Vincent Micheli, Eloi Alonso, and François Fleuret. Efficient world models with context-aware tokenization. *arXiv preprint arXiv:2406.19320*, 2024.

[26] NE-Dreamer Authors. Next embedding prediction makes world models stronger. *arXiv preprint arXiv:2603.02765*, 2026.

[27] NeSyS Authors. Neuro-symbolic synergy for interactive world modeling. *arXiv preprint arXiv:2602.10480*, 2026.

[28] Weili Nie, Zhiding Yu, Lei Mao, Ankit B. Patel, Yuke Zhu, and Anima Anandkumar. Bongard-LOGO: A new benchmark for human-level concept learning and reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

[29] Open-Ended Learning Team, Adam Stooke, Anuj Mahajan, et al. Open-ended learning leads to generally capable agents. *arXiv preprint arXiv:2107.12808*, 2021.

[30] OpenAI. Learning to reason with LLMs. OpenAI technical report, 2024.

[31] OpenAI. Video generation models as world simulators. OpenAI technical report, 2024.

[32] Wasu Top Piriyakulkij, Yichao Liang, Hao Tang, et al. PoE-World: Compositional world modeling with products of programmatic experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.

[33] Thomas M. Scanlon. *What We Owe to Each Other*. Harvard University Press, 1998.

[34] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, et al. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.

[35] James Schwoebel, Ingrida Semenec, Jenia Rousseva, Marcos Ortiz, Collin Overbay, Manish Bhatt, Rome Thorstenson, and Martin G. Frasch. OpenWorld: A zero-dependency framework for authoring, verifying, composing, and serving symbolic code world models, 2026. Companion paper.

[36] Avi Singh, John D. Co-Reyes, Rishabh Agarwal, Ankesh Anand, Piyush Patil, Xavier Garcia, Peter J. Liu, James Harrison, Jaehoon Lee, Kelvin Xu, et al. Beyond human data: Scaling self-training for problem-solving with language models. *Transactions on Machine Learning Research (TMLR)*, 2024.

[37] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.

[38] Specification Trap Authors. The specification trap: Why static value alignment alone is insufficient for robust alignment. *arXiv preprint arXiv:2512.03048*, 2025.

[39] Aarohi Srivastava et al. Beyond the imitation game: Quantifying and extrapolating the capabilities of language models. *Transactions on Machine Learning Research (TMLR)*, 2023.

[40] Hao Tang, Darren Key, and Kevin Ellis. WorldCoder, a model-based LLM agent: Building world models by writing code and interacting with the environment. *arXiv preprint arXiv:2402.12275*, 2024.

[41] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[42] Petar Veličković, Adrià Puigdomènech Badia, David Budden, Razvan Pascanu, Andrea Banino, Misha Dashevskiy, Raia Hadsell, and Charles Blundell. The CLRS algorithmic reasoning benchmark. In *International Conference on Machine Learning (ICML)*, 2022.

[43] Zhouliang Yu, Yuhuan Yuan, Tim Z. Xiao, et al. Generating symbolic world models via test-time scaling of large language models. *arXiv preprint arXiv:2502.04728*, 2025.

[44] Zheng Yuan, Hongyi Yuan, Chengpeng Li, Guanting Dong, Keming Lu, Chuanqi Tan, Chang Zhou, and Jingren Zhou. Scaling relationship on learning mathematical reasoning with large language models. *arXiv preprint arXiv:2308.01825*, 2023.

[45] Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

[46] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, et al. Judging LLM-as-a-judge with MT-Bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[47] Mingchen Zhuge, Changsheng Zhao, Dylan Ashley, et al. Agent-as-a-judge: Evaluate agents with agents. *arXiv preprint arXiv:2410.10934*, 2024.